

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH**



**BÁO CÁO ĐỒ ÁN
ĐỀ TÀI: KHÓA SỐ ĐIỆN TỬ SỬ DỤNG
FPGA**

Môn học: Thiết kế hệ thống số với HDL

Lớp CE213.Q22

Giảng viên hướng dẫn: Ths. Hồ Ngọc Diễm

Thực hiện bởi nhóm, bao gồm:

Nguyễn Đức Khang	23520695
Võ Gia Huy	23620654
Nguyễn Vương Hiếu	23520488

MỤC LỤC

MỤC LỤC	1
TÓM TẮT	3
Chương I. TỔNG QUAN	4
1. Giới thiệu đề tài	4
1.1. Bối cảnh và Lý do chọn đề tài	4
1.2. Mục tiêu đề tài	4
1.3. Phạm vi và Các tính năng nổi bật	4
1.4. Ý nghĩa thực tiễn	4
2. Cơ sở lý thuyết	4
2.1. Cơ sở lý thuyết Khối Input (Tiếp nhận & Tiền xử lý tín hiệu)	5
2.2. Cơ sở lý thuyết Khối Logic:	5
2.3. Cơ sở lý thuyết khối Output (Output Algorithms)	6
Chương II. GIỚI THIỆU CÁC THIẾT BỊ	7
1. Bàn phím OEM 4x4	7
2. Altera DE2 (Cyclone II FPGA)	9
3. LCD 16x2 trên FPGA	10
4. Buzzer 3V:	11
CHƯƠNG III: CÁC CHẾ ĐỘ VÀ LUỒNG HOẠT ĐỘNG CỦA HỆ THỐNG	12
CHƯƠNG IV : DANH SÁCH CÁC MODULE	14
1. Khối input :	14
2. Khối Logic Khóa:	14
3. Khối output:	15
CHƯƠNG V : CHI TIẾT CODE CÁC MODULE	16
1. khối xử lý Input:	16
1.1. Module Chiauxung.v	16
1.2. Module chongnhieu.v	18
1.3. Module quetkeypad.v:	20
2. Khối xử lý Logic trung tâm (Logic Control)	21
2.1. Module: shiftbuffer.v	21
2.2. Module: passdatapath.v	22
2.3. Module: fsm_controller.v	23
3. Khối xử lý Output (Output & Display):	24
3.1. Module: led_7_doan.v (Giải mã hiển thị LED)	24
Chương VI. KIỂM TRA VÀ MÔ PHỎNG HỆ THỐNG	28
1. Mục tiêu kiểm tra	28

2. Các công cụ hỗ trợ trong kịch bản (Helper Tasks).....	28
3. Kịch bản các trường hợp kiểm thử (Test Scenarios)	28
4. Mô phỏng:	29
4.1. Waveform và console:.....	29
4.2. Phân tích mô phỏng:.....	31
5. Gắn Pin:.....	36
6. Test trực tiếp trên Kit DE2:	36
CHƯƠNG VII: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	37
1. Kết quả đạt được	37
2. Đánh giá ưu điểm và hạn chế	37
2.1. Ưu điểm.....	37
2.2. Hạn chế.....	37
3. Hướng phát triển đề tài.....	37
NHẬN XÉT VÀ ĐÁNH GIÁ	38
BẢNG PHÂN CÔNG CÔNG VIỆC	38

TÓM TẮT

Đề tài tập trung nghiên cứu và thiết kế hệ thống khóa số điện tử trên nền tảng kit phát triển Altera FPGA DE2. Mục tiêu cốt lõi của đề án là xây dựng một hệ thống khóa mã pin sử dụng FPGA làm bộ xử lý trung tâm, đảm bảo sự kết hợp hài hòa giữa tính đơn giản trong vận hành, đa dạng về chức năng, cùng độ ổn định cao và giao diện thân thiện với người sử dụng.

Chương I. TỔNG QUAN.

1. Giới thiệu đề tài

1.1. Bối cảnh và Lý do chọn đề tài

Trong thời đại công nghệ số hiện nay, bảo mật thông tin và an toàn tài sản là một trong những nhu cầu thiết yếu nhất. Các hệ thống khóa số điện tử đang dần thay thế hoàn toàn khóa cơ truyền thống nhờ vào tính bảo mật cao, sự tiện dụng và khả năng tùy biến linh hoạt. Hiện nay, đa số các loại khóa trên thị trường đều sử dụng vi điều khiển (Microcontroller).

Tuy nhiên, việc ứng dụng công nghệ linh kiện logic có thể lập trình - FPGA (Field Programmable Gate Array) - mở ra một hướng tiếp cận mạnh mẽ hơn: khả năng xử lý song song tốc độ cao, độ ổn định tuyệt đối của mạch cứng và khả năng tái cấu trúc dễ dàng mà không bị phụ thuộc vào các tập lệnh tuần tự như CPU. Xuất phát từ thực tiễn đó, đề tài "Nghiên cứu và Thiết kế Hệ thống Khóa số trên nền tảng Altera FPGA DE2" đã được lựa chọn để thực hiện.

1.2. Mục tiêu đề tài

Mục tiêu cốt lõi của đồ án là tự thiết kế và tổng hợp một hệ thống khóa mã pin sử dụng FPGA làm bộ xử lý trung tâm. Hệ thống hướng tới việc cân bằng giữa cấu trúc logic phần cứng chặt chẽ bên trong và trải nghiệm tương tác thân thiện, mượt mà cho người dùng ở bên ngoài.

1.3. Phạm vi và Các tính năng nổi bật

Hệ thống được thiết kế hoàn toàn bằng ngôn ngữ mô tả phần cứng Verilog, chia thành các module độc lập (chia để trị) và được điều phối bởi một Máy trạng thái hữu hạn (FSM). Sản phẩm hoàn thiện trên board mạch Altera DE2 bao gồm các tính năng chính:

- **Thao tác linh hoạt:** Hỗ trợ nhập mã PIN 4 chữ số, chức năng xóa lùi (Backspace) nhanh chóng khi nhập sai hoặc xóa trắng (Clear) để nhập lại từ đầu.
- **Bảo mật nâng cao:** Tích hợp bộ mã hóa dữ liệu (Datapath) sử dụng kỹ thuật XOR Masking để xáo trộn và bảo vệ mật khẩu gốc, cho phép người dùng tự đổi mật khẩu mới một cách an toàn.
- **Giao diện trực quan:** Hiển thị trạng thái hệ thống rõ ràng thông qua hệ thống LED (LED 7 đoạn, LED đơn) và phát tín hiệu cảnh báo (còi hú) khi nhập sai quá số lần quy định.

1.4. Ý nghĩa thực tiễn

Đề tài không chỉ là cơ hội để vận dụng và củng cố các kiến thức chuyên ngành về thiết kế vi mạch số (Digital Design) và ngôn ngữ Verilog, mà còn tạo ra một sản phẩm phần cứng có độ hoàn thiện cao, hoạt động ổn định và hoàn toàn có thể phát triển thêm để ứng dụng vào các hệ thống nhà thông minh (Smart Home) hoặc kết sắt bảo mật trong thực tế.

2. Cơ sở lý thuyết

Các công nghệ và kỹ thuật cốt lõi được triển khai trong đồ án bao gồm:

2.1. Cơ sở lý thuyết Khối Input (Tiếp nhận & Tiền xử lý tín hiệu)

Giai đoạn này áp dụng các nguyên lý xử lý tín hiệu số cơ bản để giao tiếp với phần cứng cơ học:

- **Định thời & Quét ma trận (Pmod Keypad 4x4):** Sử dụng bộ đếm để chia tần số đồng bộ, kết hợp kỹ thuật Quét đa hợp phân thời gian để giao tiếp với bàn phím ma trận 4x4 (như Pmod Keypad). Hệ thống tuần tự cấp logic thấp (0) vào 4 chân hàng (Rows) và đọc trạng thái từ 4 chân cột (Columns), giúp quét toàn bộ 16 phím bấm vật lý chỉ với 8 chân I/O.
- **Khử nhiễu & Giải mã:** Ứng dụng Bộ lọc thông thấp số và Phát hiện sườn xung đồng bộ (D-Flip Flop) để loại bỏ nhiễu cơ học (Bouncing) đặc trưng của nút nhấn; sau đó dùng Mạch logic tổ hợp (Bảng chân trị) để ánh xạ lưới tọa độ vật lý (Hàng-Cột) của Pmod Keypad thành chuẩn dữ liệu nhị phân 4-bit (Mã Hex).

2.2. Cơ sở lý thuyết Khối Logic:

2.2.1. Nhóm Thuật toán Tiền xử lý tín hiệu (Input Algorithms)

- **Thuật toán Quét ma trận phím (Matrix Multiplexing):** Thuật toán dùng để nội suy ra giá trị thập phân của phím được nhấn dựa trên tọa độ lưới. Giả sử R_i là chỉ số hàng ($i = 0, 1, 2, 3$) và C_j là chỉ số cột ($j = 0, 1, 2, 3$), công thức tính toán mã phím K cơ bản là:

$$K = (4 \times R_i) + C_j$$

- (Trong thực tế thiết kế phần cứng, hệ thống sử dụng mạch giải mã tổ hợp (Decoder) để ánh xạ trực tiếp tổ hợp bit tọa độ thành mã Hex 4-bit tương ứng nhằm tối ưu tài nguyên).
- **Thuật toán Lọc nhiễu kỹ thuật số (Digital Debouncing Algorithm):** Sử dụng nguyên lý lấy mẫu theo chu kỳ (Sampling). Tín hiệu đầu vào S chỉ được công nhận là ổn định (mức 1) nếu nó duy trì mức logic 1 liên tục trong N chu kỳ lấy mẫu liên tiếp. Biểu thức logic (phép AND tích lũy) được biểu diễn như sau:

$$S_{\text{stable}} = S(t) \text{ AND } S(t - 1) \text{ AND } S(t - 2) \text{ AND } \dots \text{ AND } S(t - N)$$

- Trong đó, khoảng cách giữa các lần lấy mẫu là chu kỳ xung clock chia tần, ví dụ 10ms.

2.2.2. Nhóm Thuật toán Xử lý cốt lõi (Core Logic & Datapath)

- **Thuật toán Dịch bit (Bit Shifting Algorithm):** Dùng để cập nhật mật khẩu khi người dùng nhấn một phím mới (dữ liệu dịch từ phải sang trái). Gọi mảng dữ liệu 16-bit hiện tại là D_{old} và 4-bit phím mới nhập là I_{new} , phép dịch trái 4 bit được biểu diễn bằng biểu thức: $D_{\text{new}} = (D_{\text{old}} \ll 4) \text{ OR } I_{\text{new}}$ (Trong ngôn ngữ Verilog, công thức này được thực thi đồng thời nhờ toán tử ghép mảng $\{D_{\text{old}}[11:0], I_{\text{new}}\}$, không gây độ trễ như vòng lặp phần mềm).
- **Công thức Mã hóa và Xáo trộn dữ liệu (XOR Masking Encryption):** Đây là

thuật toán mật mã học đối xứng (Symmetric-key algorithm) đơn giản nhưng cực kỳ hiệu quả đối với tài nguyên phần cứng. Mật khẩu không được lưu trữ ở dạng văn bản thô (Plaintext) mà bị xáo trộn bởi một mặt nạ (Mask/Key).

- Khi mã hóa (Lưu mật khẩu mới):

$$C = P \oplus M$$

- Khi giải mã (Xác thực dữ liệu): $P = C \oplus M$ (Trong đó: C là Ciphertext - dữ liệu đã mã hóa lưu trong thanh ghi; P là Plaintext - mật khẩu gốc 16-bit; M là Mask - một chuỗi khóa ngẫu nhiên cố định trong hệ thống; và $\oplus$ là toán tử logic bộ cổng XOR).
- **Thuật toán So sánh Số học (Digital Equality Comparator):** Để đối chiếu 16-bit mật khẩu người dùng nhập (A) với 16-bit mật khẩu gốc (B), hệ thống sử dụng cổng XNOR (so sánh cân bằng từng bit) kết hợp với cây logic AND thay vì dùng bộ trừ số học:

$$\text{Match} = (A_0 \text{ XNOR } B_0) \text{ AND } (A_1 \text{ XNOR } B_1) \text{ AND } \dots \text{ AND } (A_15 \text{ XNOR } B_15)$$

(Kết quả Match = 1 khi và chỉ khi toàn bộ 16 cặp bit tương ứng của A và B hoàn toàn trùng khớp).

2.3. Cơ sở lý thuyết khối Output (Output Algorithms)

- **Thuật toán Giải mã BCD sang 7 đoạn (BCD to 7-Segment Decoding):** Ánh xạ dữ liệu nhị phân 4-bit (từ 0000 đến 1001) thành dải 7-bit (a, b, c, d, e, f, g) để điều khiển tắt/mở các thanh LED.
- Hệ thống sử dụng các phương trình tối giản Karnaugh (K-map) để thiết kế mạch tổ hợp. Ví dụ phương trình mức cổng cho thanh sáng 'a' (tích cực mức thấp) có dạng logic: $a = (\text{NOT } A3 \text{ AND NOT } A2 \text{ AND NOT } A1 \text{ AND } A0) \text{ OR } (\text{NOT } A3 \text{ AND } A2 \text{ AND NOT } A1 \text{ AND NOT } A0)$ (Trong đó A3, A2, A1, A0 là các bit tương ứng của một chữ số BCD).

Chương II. GIỚI THIỆU CÁC THIẾT BỊ

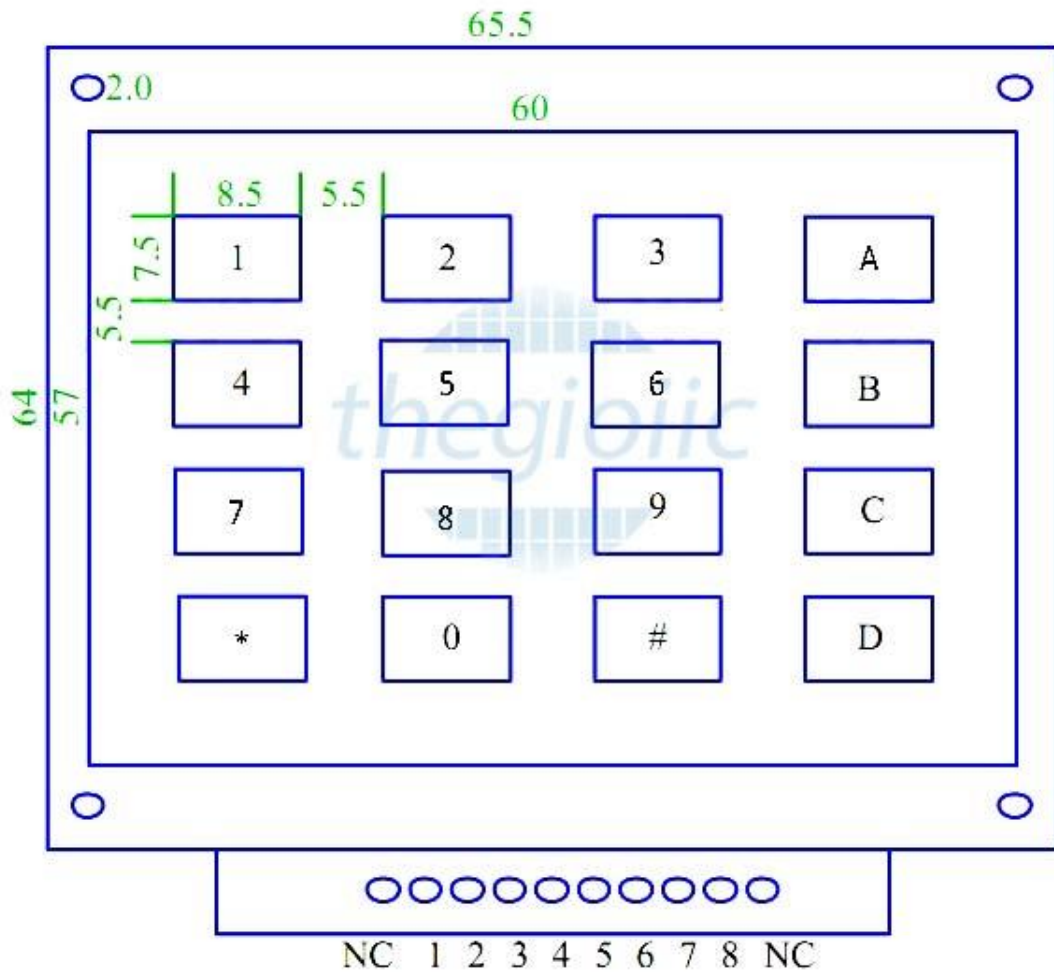
1. Bàn phím OEM 4x4

Bàn phím ma trận 4x4 (Pmod Keypad) là thiết bị ngoại vi đầu vào cốt lõi, chịu trách nhiệm tiếp nhận dữ liệu thao tác trực tiếp từ người dùng.

- Cấu tạo vật lý: Bao gồm 16 nút nhấn cơ học (từ 0-9, A-D, *, #) được sắp xếp thành lưới 4 hàng và 4 cột. Bề mặt phím hành trình 1.5mm, lực nhấn 1.18N cùng tuổi thọ lên đến 2 triệu lần thao tác.
- Giao tiếp ma trận (Matrix Interface): Đây là đặc điểm quan trọng nhất giúp thiết bị tối ưu hóa không gian. Thay vì cần 16 đường tín hiệu độc lập cho 16 phím, hệ thống sử dụng cáp bus 8 chân (4 chân hàng, 4 chân cột) kết hợp kỹ thuật quét đa hợp để giải mã tọa độ.



OEM 4x4 keypad



Sơ đồ chân

1(1,5) 2(2,5) 3(3,5) A(4,5)

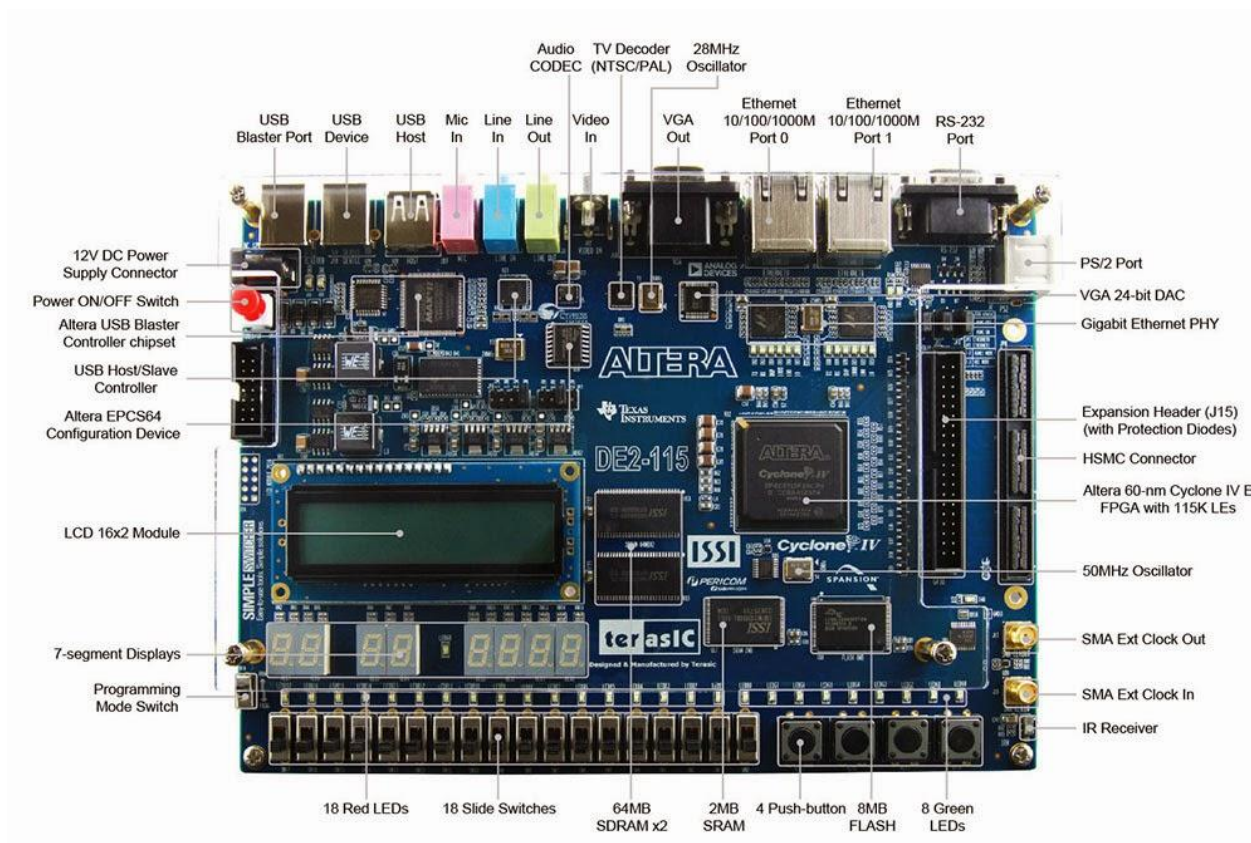
4(1,6) 5(2,6) 6(3,6) B(4,6)

7(1,7) 8(2,7) 9(3,7) C(4,7)

*(1,8) 0(2,8) #(3,8) D(4,8)

Sơ đồ ma trận chân trên phím

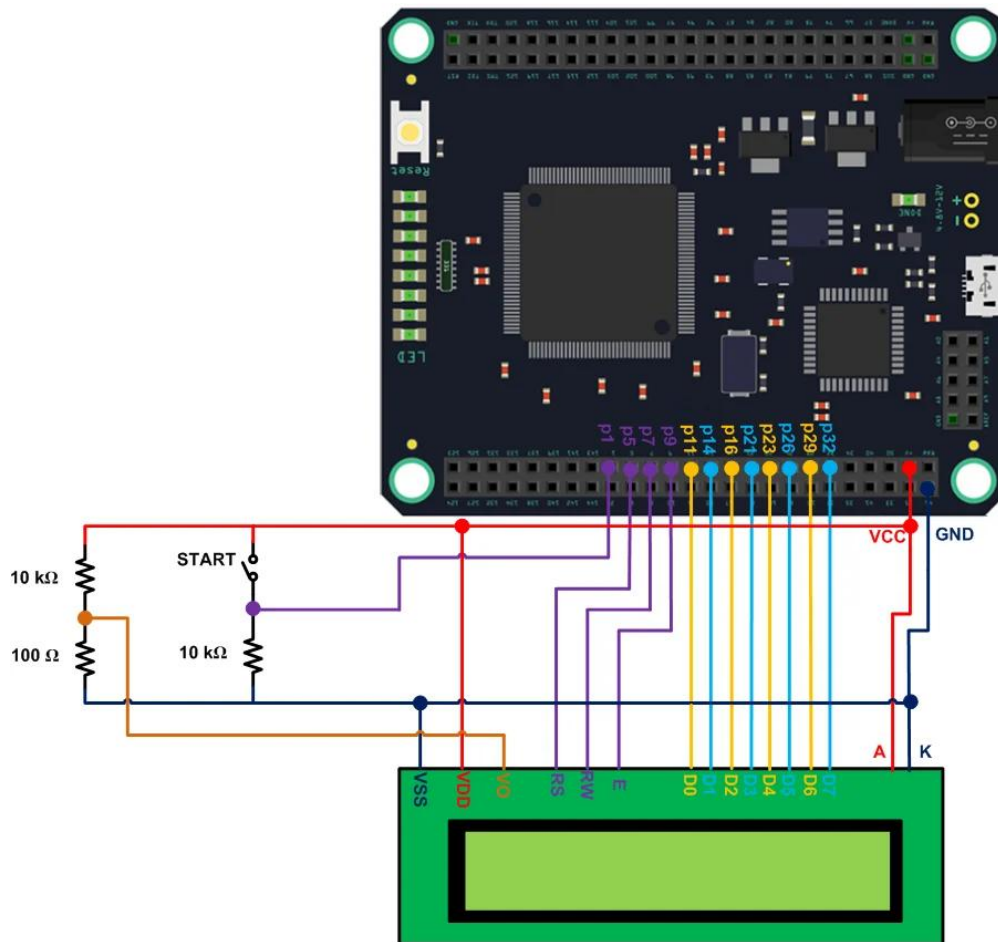
2. Altera DE2 (Cyclone II FPGA)



Altera Cyclone II (trên kit DE2) là bộ não trung tâm của hệ thống, thuộc dòng chip logic khả trình (FPGA) tiêu chuẩn của hãng Altera (Intel).

- **Kiến trúc lõi:** Mạng cổng logic lập trình được với 33.216 phần tử logic (Logic Elements) hoạt động đồng bộ ở xung nhịp gốc 50 MHz.
- **Xử lý song song (True Parallel Processing):** Đây là đặc điểm quan trọng nhất giúp FPGA thực thi đồng thời nhiều khối thuật toán (như quét ma trận phím, so sánh mật khẩu XOR và xuất tín hiệu màn hình) cực nhanh mà không bị hiện tượng thất cổ chai hay trễ thời gian như các vòng lặp tuần tự của vi điều khiển.
- **Tài nguyên:** Chip tích hợp sẵn bộ nhân chia tần số (PLL), mạng lưới thanh ghi (Registers) dồi dào để xây dựng Máy trạng thái (FSM), và số lượng lớn chân I/O linh hoạt để giao tiếp trực tiếp với đa dạng thiết bị ngoại vi cùng lúc.

3. LCD 16x2 trên FPGA



Màn hình LCD 16x2 (Driver HD44780) đóng vai trò là giao diện hiển thị trực quan (UI), chịu trách nhiệm phản hồi mọi trạng thái hoạt động của hệ thống khóa số đến người dùng.

- **Giao tiếp phân cứng:** Thiết bị nhận tín hiệu trực tiếp từ board mạch FPGA thông qua chuẩn giao tiếp song song. Quá trình truyền dữ liệu sử dụng các chân điều khiển logic (RS để chọn thanh ghi lệnh/dữ liệu, R/W để ghi, E để chốt tín hiệu) kết hợp cùng bus dữ liệu 8-bit (DB0-DB7) để truyền mã ASCII.
- **Điều phối hiển thị (Control Logic):** LCD hoạt động dưới sự kiểm soát của module lcd_controller.v. Khi Máy trạng thái (FSM) của hệ thống chuyển đổi (ví dụ: đang chờ nhập, mở khóa thành công, báo lỗi sai mã), module điều khiển sẽ dịch các trạng thái này thành mã lệnh, tuân thủ nghiêm ngặt giản đồ thời gian (timing diagram) của chip HD44780 để in chuỗi ký tự ra màn hình.
- **Trải nghiệm người dùng:** Với khả năng hiển thị 32 ký tự (16 cột x 2 dòng) cùng đèn nền độ tương phản cao, LCD liên tục cập nhật các thông điệp chỉ dẫn rõ ràng (ví dụ: "ENTER PIN", "ACCESS GRANTED", "WRONG PIN"), giúp hệ thống khóa số trở nên chuyên nghiệp và dễ dàng tương tác.

4. Buzzer 3V:

Active Buzzer



Passive Buzzer



- Kết nối vật lý (GPIO):
 - Chân GND: Nối với cực âm chung của cổng GPIO.
 - Chân Signal: Nhận tín hiệu điều khiển (Warning) từ chân I/O của FPGA.
- Cơ chế hoạt động:
 - Kích hoạt khi hệ thống chuyển sang trạng thái báo động (nhập sai 3 lần).
- Phương thức điều khiển:
 - Active Buzzer: Xuất mức logic cao (1) để hú còi liên tục.
 - Passive Buzzer: Xuất chuỗi xung vuông để tạo tần số âm thanh.

CHƯƠNG III: CÁC CHẾ ĐỘ VÀ LUỒNG HOẠT ĐỘNG CỦA HỆ THỐNG

DANH SÁCH CÁC CHỨC NĂNG CỦA HỆ THỐNG

- **Mở khóa an toàn (Secure Authentication):**

Hệ thống cho phép người dùng nhập mật khẩu bảo mật gồm 4 chữ số. Dữ liệu đầu vào được che giấu chặt chẽ thông qua thuật toán xáo trộn (XOR Masking) trước khi tiến hành đối chiếu với chuỗi mật khẩu gốc để cấp quyền truy cập.

- **Cập nhật mật khẩu bảo mật (Secure Password Update):**

Nhằm đảm bảo tính toàn vẹn của hệ thống, cơ chế phân luồng yêu cầu người dùng phải xác thực chính xác mật khẩu hiện tại trước khi cấp quyền định tuyến sang trạng thái thiết lập và ghi đè mật khẩu mới vào bộ nhớ.

- **Hiệu chỉnh dữ liệu nhập (Typo Correction/Backspace):**

Tích hợp phím chức năng chuyên biệt cho phép người dùng thao tác xóa lùi (Backspace) từng ký tự. Tính năng này giúp tối ưu hóa trải nghiệm, cho phép khắc phục các lỗi gõ sai trong quá trình nhập liệu một cách linh hoạt mà không cần phải hủy bỏ chu trình thao tác.

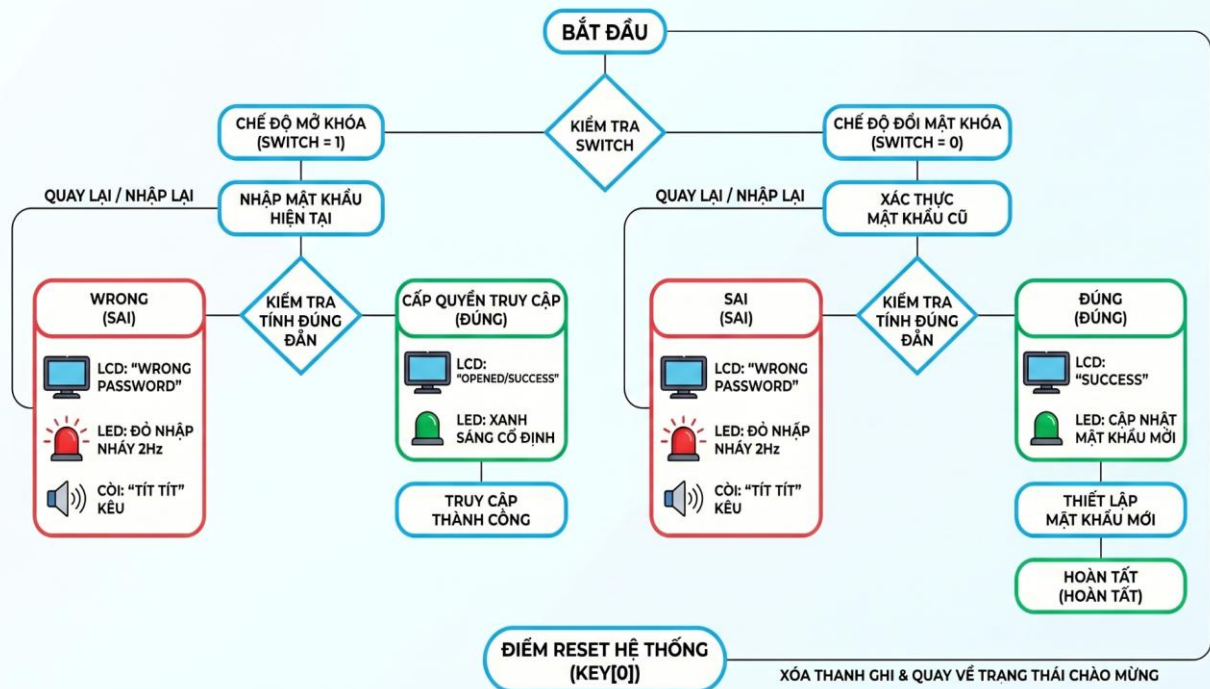
- **Cảnh báo an ninh ngoại vi (Security Alert System):**

Khi phát hiện hành vi nhập sai mật khẩu 3 lần, hệ thống sẽ ngay lập tức kích hoạt cơ chế báo động. Cơ chế này bao gồm tín hiệu âm thanh định thời từ còi (Buzzer) kết hợp cùng hiệu ứng nhấp nháy từ LED Đỏ, cảnh báo trực quan các nỗ lực xâm nhập trái phép.

- **Khôi phục toàn cục (Global Reset):**

Cung cấp cơ chế can thiệp phần cứng khẩn cấp, cho phép đưa toàn bộ hệ thống (bao gồm Máy trạng thái, các thanh ghi đệm và thiết bị ngoại vi) về trạng thái an toàn ban đầu tại bất kỳ thời điểm nào, giúp ngăn chặn triệt để tình trạng treo mạch (system hang) trong quá trình vận hành.

Quy Trình Hoạt Động Hệ Thống Khóa Số Thông Minh (FPGA Altera DE2)



CHƯƠNG IV : DANH SÁCH CÁC MODULE

1. Khối input :

Tên Module	Chức năng chính trong hệ thống
chia_xung.v	Nhận xung clock 50MHz từ FPGA và chia tần số để tạo ra các xung nhịp chuẩn: 1kHz (cấp cho mạch quét phím) và 100Hz (cấp cho mạch lọc nhiễu)
Quetkeypad.v	Xuất tín hiệu quét lần lượt các hàng, đọc tín hiệu cột phản hồi để phát hiện vị trí phím nhấn, đồng thời giải mã tọa độ này thành giá trị thập lục phân (hex_val từ 0-F).
Chongnhieu.v	Sử dụng thanh ghi dịch để lọc hiện tượng dội phím cơ học (bouncing). Phát hiện sườn tín hiệu để tạo ra một xung cờ báo (co_phim) duy nhất, tránh lỗi nhận nhiều lần cho 1 lần nhấn.
top_input.v	Khối giao tiếp cấp cao nhất. Liên kết và đồng bộ tín hiệu giữa 3 module con trên, nhận tín hiệu thô từ phần cứng và xuất dữ liệu sạch (đã giải mã và lọc nhiễu) cho hệ thống xử lý trung tâm.

2. Khối Logic Khóa:

Tên Module	Chức năng chính trong hệ thống
fsm_controller.v	Nhận tín hiệu phím bấm để chuyển đổi qua lại giữa 6 trạng thái: Chờ, Nhập, Kiểm tra, Mở cửa, Báo lỗi, Đổi mật khẩu. Tự động xuất lệnh điều khiển các khối dữ liệu. Có tích hợp bộ đếm định thời (timer) và đếm lỗi.
Shiftbuffer.v	Chờ lệnh từ FSM để thực hiện: Dịch phím mới vào từ bên phải (nhập số), xóa lùi 1 ký tự, xóa trắng toàn bộ 16-bit về 0.
Passdatapath.v	Lưu trữ mật khẩu gốc được xáo trộn bằng một lớp mặt nạ (XOR Masking để chống đọc ngược). Nhận lệnh từ FSM để so sánh mật khẩu nhập với kết quả, hoặc cập nhật mật khẩu mới.
logic_top.v	Đóng gói toàn bộ 3 module con bên dưới thành một khối duy nhất.

3.Khối output:

Tên Module	Chức năng chính trong hệ thống
led_7_doan.v	Tách mảng dữ liệu 16-bit thành 4 cụm 4-bit độc lập. Giải mã từng cụm thành mã hiển thị 7 đoạn (chuẩn Anode chung). Đặc biệt, xử lý mã ngoại lệ 4'hF thành lệnh xuất 7'b1111111 để tắt hoàn toàn các bóng LED tại vị trí người dùng chưa nhập số.
warning_buzzer.v	Đọc các cờ trạng thái từ FSM để điều khiển thiết bị báo hiệu. Sử dụng bộ chia tần số từ xung clock 50MHz để tạo ra các hiệu ứng: Sáng tĩnh LED Xanh khi mở cửa thành công; và nhấp nháy LED Đỏ kèm Còi hú đồng bộ ở tần số 2Hz khi nhập sai mật khẩu.
lcd_controller.v	Nhận mã state_code từ FSM để tự động chọn chuỗi ký tự ASCII tương ứng (ví dụ: "ENTER PASSWORD"). Sinh ra các xung tín hiệu điều khiển (đặc biệt là xung LCD_EN) chuẩn timing của dòng IC HD44780 để in chữ mượt mà, không rác. Cố định chân LCD_RW ở mức 0.
output_top.v	Đóng gói (wrap) cả 3 module (led_7_doan, warning_buzzer, lcd_controller) lại thành một khối tổng duy nhất. Đi dây nội bộ để đồng bộ tín hiệu và gom toàn bộ các ngõ ra để kết nối trực tiếp với chân vật lý (pin) của board FPGA

CHƯƠNG V : CHI TIẾT CODE CÁC MODULE

1. khối xử lý Input:

1.1. Module Chiaxung.v

```
module chiaxung(  
    input clk,  
    input reset_n,  
    output tick_1000hz,  
    output tick_100hz,  
    output tick_1sec
```

- **input**

- clk : xung nhịp gốc thạch anh của fpga là 50MHZ
- reset_n : nút reset cho tất cả các giá trị về lại 0, thời gian đếm về lại 0 , tích cực mức thấp

- **output :**

- Tick_1000hz : tích các xung cạnh lên trong thời gian 1ms , dùng để kích đoạn led 7 đoạn
- Tick_100hz : tích các xung cạnh lên trong thời gian 10ms , dùng để chống dôi các phím cơ học trên bàn phím pmod lọc nhiễu để
- Tick_1s: tích các xung cạnh lên trong thời gian 1 giây thời gian thực dùng để đếm thời gian thực tạo hiệu ứng nhảy đèn

Các thanh ghi:

```
reg[15:0] count_1000hz;  
reg[18:0] count_100hz;  
reg[25:0] count_1sec;
```

- **Thanh ghi count_1000hz:**

- Để tạo ra tần số 100hz , bộ đếm cần đếm $50M/1000 = 50k$ nhịp $\Rightarrow$ (0 tới 49,999) ta sử dụng $2^{16} - 1 = 65,535$ (đủ khả năng chứa 49,999)

- **Thanh ghi count_100hz**

- Để tạo ra tần số 100hz , bộ đếm cần đếm $50M/100 = 500k$ nhịp $\Rightarrow$ (0 tới 499,999) ta sử dụng $2^{19} - 1 = 524,287$ (đủ khả năng chứa 499,999)

- **Thanh ghi count_1s**

- Để tạo ra tần số 1hz , bộ đếm cần đếm $50M/1 = 50M$ nhịp $\Rightarrow$ (0 tới 49,999,999) ta sử dụng $2^{26} - 1 = 67,108,863$ (đủ khả năng chứa 50M)

```

always@(posedge clk or negedge reset_n)
begin
    if(!reset_n)
    begin
        count_1000hz <= 16'd0;
        count_100hz <= 19'd0;
        count_1sec <= 26'd0;
    end
    else begin
        if(count_1000hz ==16'd49_999)
        count_1000hz <= 16'd0;
        else count_1000hz <= count_1000hz + 16'd1;

        if(count_100hz ==19'd499_999)
        count_100hz <= 19'd0;
        else
        count_100hz <= count_100hz + 19'd1;

        if(count_1sec == 26'd49_999_999)
        count_1sec <= 26'd0;
        else
        count_1sec <= count_1sec +26'd1;
    end
end
assign tick_1000hz =(count_1000hz==16'd49_999);
assign tick_100hz = (count_100hz==19'd499_999);
assign tick_1sec = (count_1sec==26'd49_999_999);

endmodule

```

Nguyên lý hoạt động chi tiết:

- **Cơ chế Reset:** Sử dụng tín hiệu reset không đồng bộ mức thấp (negedge reset_n) để lập tức xóa toàn bộ thanh ghi đếm về 0, đưa hệ thống về trạng thái bắt đầu.
- **Cơ chế Đếm:** Đồng bộ theo sườn lên xung nhịp (posedge clk), các thanh ghi đếm tăng liên tục và tự động quay vòng về 0 khi chạm các ngưỡng thiết lập: 49_999 (chu kỳ 1ms), 499_999 (10ms), và 49_999_999 (1s).
- **Cơ chế Tạo Xung (Tick):** Thông qua lệnh assign, ngõ ra chỉ nảy lên mức 1 chớp nhoáng ngay tại khoảnh khắc bộ đếm đạt giá trị đỉnh. Hệ quả là mạch tạo ra một xung cực hẹp (rộng đúng 1 chu kỳ clock), giúp đồng bộ cho các module phía sau.

1.2. Module chongnhieu.v

```
module chongnhieu (
    input clk,
    input reset_n,
    input tick_100,
    input [3:0] col_data,
    output co_phim
);

    wire any_col_active = &col_data;
```

- **clk & reset_n:**
 - Xung nhịp hệ thống (50MHz) và tín hiệu xóa (reset) tích cực mức thấp để khởi tạo lại toàn bộ mạch.
- **tick_100:**
 - Xung lấy mẫu (Sample Tick). Đây chính là tín hiệu chu kỳ 10ms (100Hz) được cấp từ module chia xung trước đó. Nó làm nhiệm vụ "nhắc nhịp" để mạch tiến hành kiểm tra trạng thái phím sau mỗi 10ms, giúp bỏ qua các rung động nhiễu chớp nhoáng của lò xo cơ học.
- **col_data [3:0]:**
 - Dữ liệu 4 cột quét từ bàn phím ma trận (hoạt động ở mức thấp - Active Low). Lệnh `wire any_col_active = &col_data;` dùng cổng AND gom 4 bit này lại:
 - Nếu không bấm phím (Cả 4 bit là 1111) -> Kết quả = 1.
 - Nếu có bấm phím (Có ít nhất 1 bit 0, ví dụ 1101) -> Kết quả = 0.
- **co_phim (Output):**
 - Cờ báo hiệu có phím được bấm. Tín hiệu này chỉ nẩy lên mức 1 chớp nhoáng (đúng 1 chu kỳ clock) ngay tại khoảnh khắc một thao tác bấm phím hợp lệ và sạch nhiễu được ghi nhận.

Đồng bộ tín hiệu (ff1, ff2)

```
reg ff1, ff2;
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        ff1 <= 1'b1;
        ff2 <= 1'b1;
    end else begin
        ff1 <= any_col_active;
        ff2 <= ff1;
    end
end
```

- **Vai trò:**
 - Chống trạng thái lơ lửng (Metastability). Tín hiệu từ nút bấm vật lý đi vào là không đồng bộ (Asynchronous) so với xung clk của FPGA. Nếu đưa trực tiếp vào tính toán có thể gây lỗi logic.
- **Hoạt động:**

- Hai thanh ghi ff1 và ff2 mắc nối tiếp tạo thành bộ "2-Stage Synchronizer". Tín hiệu phím thô sẽ được chốt qua 2 nhịp clock để đảm bảo nó trở thành một tín hiệu số 0 hoặc 1 dứt khoát trước khi đưa vào xử lý.

Bộ đệm lịch sử lấy mẫu (shift_reg)

```
reg [1:0] shift_reg;
reg btn_stable;

always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        shift_reg <= 2'b11;
    end
    else if (tick_100) begin
        shift_reg <= {shift_reg[0], ff2};
    end
end
```

- **Vai trò:**
 - Ghi nhớ trạng thái phím trong 2 chu kỳ lấy mẫu liên tiếp.
- **Hoạt động:**
 - Thanh ghi dịch shift_reg [1:0] chỉ hoạt động khi có xung tick_100 (mỗi 10ms). Nó dịch bit trạng thái hiện tại (ff2) vào chuỗi. Nghĩa là shift_reg sẽ chứa lịch sử trạng thái phím của 20ms vừa qua.

Bộ lọc quyết định (btn_stable)

```
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        btn_stable <= 1'b1;
    end else begin
        if (shift_reg == 2'b00)
            btn_stable <= 1'b0;
        else if (shift_reg == 2'b11)
            btn_stable <= 1'b1;
        end
    end
end
```

- **Vai trò:**
 - Loại bỏ hoàn toàn nhiễu dội (Debounce).
- **Hoạt động:**
 - Mạch chỉ công nhận là **phím đang được bấm dứt khoát** (btn_stable <= 0) nếu shift_reg == 2'b00 (nghĩa là trong suốt 20ms qua, phím liên tục ở mức 0).
 - Mạch chỉ công nhận là **phím đã được nhả ra hoàn toàn** (btn_stable <= 1) nếu shift_reg == 2'b11 (trong suốt 20ms qua, phím liên tục ở mức 1).
 - Nếu shift_reg là 01 hoặc 10 (trạng thái nhiễu, đang rung lắc), mạch sẽ "mặc kệ" và giữ nguyên trạng thái cũ.

Bộ dò xung (btn_stable_delay và lệnh assign co_phim)

```
reg btn_stable_delay;
always @(posedge clk or negedge reset_n) begin
    if (!reset_n)
        btn_stable_delay <= 1'b1;
    else
        btn_stable_delay <= btn_stable;
end

assign co_phim = btn_stable_delay & ~btn_stable;

endmodule
```

- **Vai trò:**
 - Chuyển đổi trạng thái phím dài thành một xung (pulse) duy nhất.
- **Hoạt động:**
 - Thanh ghi btn_stable_delay làm nhiệm vụ lưu lại trạng thái của btn_stable ở chu kỳ clock ngay trước đó.
- **Lệnh assign co_phim = btn_stable_delay & ~btn_stable;**
 - Đây là thuật toán tách xung sườn xuống (Falling Edge Detector). Nó tìm kiếm chính xác khoảng khắc tín hiệu chuyển từ 1 (chưa bấm) sang 0 (đã bấm) để kích hoạt co_phim = 1 trong duy nhất 1 chu kỳ clock. Nhờ vậy, dù người dùng có đè giữ phím bao lâu đi nữa, hệ thống cũng chỉ đếm là 1 lần bấm.

1.3. Module quetkeypad.v:

```
module quetkeypad(
    input clk,
    input reset_n,
    input tick_1000hz,
    input [3:0] col,
    output reg [3:0] row,
    output [3:0] col_data,
    output reg [3:0] hex_val
);
```

Đầu vào (Input):

- **clk & reset_n:** Xung nhịp hệ thống (50MHz) và tín hiệu xóa toàn mạch (tích cực mức thấp).
- **tick_1000hz:** Xung kích hoạt chu kỳ 1ms (do module chia xung cấp). Đây là tốc độ quét bàn phím lý tưởng, đủ nhanh để không lỡ thao tác của người dùng, nhưng không quá nhanh để gây gánh nặng cho vi mạch.
- **col [3:0]:** Tín hiệu đọc về từ 4 chân Cột của bàn phím. Do bàn phím ma trận thường được kéo lên VCC (Pull-up) bằng điện trở, trạng thái mặc định của các cột là 4'b1111. Khi một phím được nhấn, nó nối ngắn mạch Cột với Hàng đang quét (mức 0), làm một bit trong col bị kéo xuống 0 (VD: 4'b1101).

Đầu ra (Output):

- **row [3:0]:** Tín hiệu điều khiển cấp cho 4 chân Hàng của bàn phím. Chỉ có 1 hàng duy nhất được kéo xuống mức 0 tại một thời điểm (VD: 4'b1101 nghĩa là đang cấp điện cho Hàng 2).
- **col_data [3:0]:** Ngõ ra nối dài của col, dùng để chuyển tiếp tín hiệu thô nguyên bản sang module chongnhiều nhằm phục vụ việc phát hiện thời điểm bấm phím.
- **hex_val [3:0]:** Giá trị mã hóa (từ 0 đến F) của phím tương ứng với vị trí bị nhấn. Mã này sẽ được gửi về khối FSM để xử lý logic mật khẩu.

2. Khối xử lý Logic trung tâm (Logic Control)

2.1. Module: shiftbuffer.v

- **Vai trò khối:** Đóng vai trò là bộ đệm tạm thời (Buffer Datapath), chịu trách nhiệm lưu trữ và định hình mật khẩu 16-bit (gồm 4 ký tự mã Hex) do người dùng đang nhập vào từ bàn phím.
- **Chức năng từng khối logic:** Module này có chức năng cập nhật chuỗi số theo 3 thao tác: Dịch trái để nạp số mới, dịch phải để xóa lùi (Backspace), và xóa toàn bộ (Clear).
- **Đoạn code phân tích:**

```
module shiftbuffer(  
    input clk,  
    input reset_n,  
    input [3:0] ma_phim,  
    input co_phim,  
    input shift_en,  
    input backspace_en,  
    input clear_buf,  
    output reg [15:0] pass_16bit  
);  
    always@(posedge clk or negedge reset_n) begin  
        if(!reset_n) pass_16bit <= 16'hFFFF;  
        else if(clear_buf) pass_16bit <= 16'hFFFF;  
        else if(backspace_en) pass_16bit <= {4'hF, pass_16bit[15:4]};  
        else if (shift_en) pass_16bit <= {pass_16bit[11:0], ma_phim};  
    end  
endmodule
```

- **Giải thích chi tiết:**
 - **Xóa bộ đệm:** Mặc định khi Reset mạch hoặc khi có lệnh clear_buf từ FSM, chuỗi mật khẩu được gán bằng 16'hFFFF. Giá trị F được sử dụng như một cờ (flag) rỗng, báo cho mạch hiển thị biết rằng "chưa có số nào được nhập".
 - **Thao tác xóa lùi (backspace_en):** Sử dụng toán tử ghép mảng (concatenation), mạch đẩy 12 bit cao xuống vị trí thấp (pass_16bit[15:4]), đồng thời chèn mã 4'hF vào vị trí cao nhất.
 - **Thao tác dịch phím (shift_en):** Khi người dùng bấm phím, 4-bit ma_phim mới sẽ được chèn vào bên phải (vị trí thấp nhất), đồng thời đẩy

12 bit cũ sang trái pass_16bit[11:0].

2.2. Module: passdatapath.v

- **Vai trò khối:** Đây là "kết sắt số" của hệ thống, lưu trữ mật khẩu gốc và thực hiện các thuật toán đối chiếu bảo mật ở mức phần cứng.
- **Chức năng từng khối logic:** Lưu trữ mật khẩu mới, dùng kỹ thuật mặt nạ (XOR Masking) để xáo trộn dữ liệu, và thực hiện phép so sánh luận lý tổ hợp cực nhanh.
- **Đoạn code phân tích:**

```
localparam mask = 16'h9554;  
localparam matkhau_macdinh= 16'h0000;
```

Ghi nhận và xáo trộn mật khẩu (XOR Masking):

```
always@(posedge clk or negedge reset_n) begin  
    if (!reset_n) stored_pass <= matkhau_macdinh ^ mask;  
    else if (cmd_capnhat) stored_pass <= pass_16bit ^ mask;  
end
```

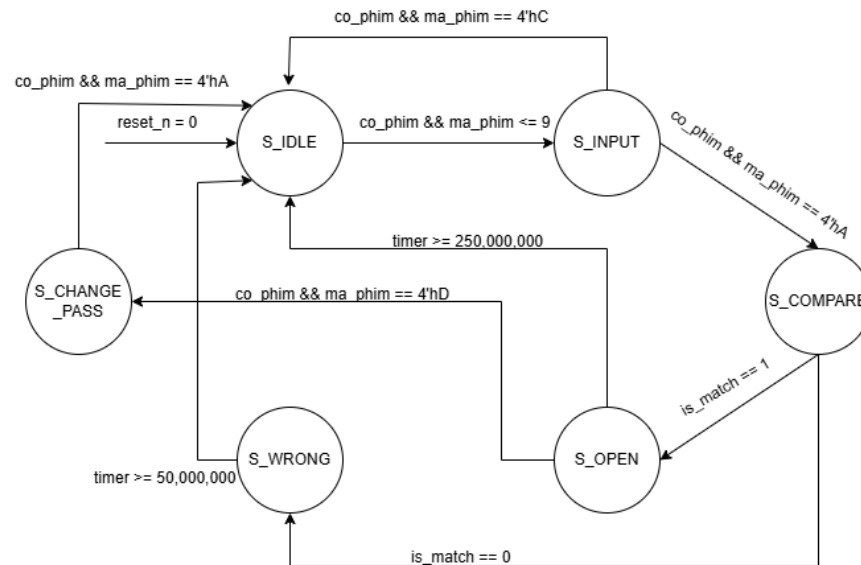
Mạch so sánh (Comparator):

```
assign trungkhop = cmd_sosanh & ((pass_16bit ^ mask) == stored_pass);
```

- **Giải thích chi tiết:**
 - **Thuật toán XOR Masking (^):** Mật khẩu không bao giờ được lưu thẳng. Mật khẩu mặc định (0000) và mật khẩu mới (pass_16bit) đều phải XOR qua một lớp mặt nạ cố định 16'h9554. Việc này chống lại việc giải mã ngược trực tiếp từ thanh ghi.
 - **Phép so sánh (assign trungkhop):** Tín hiệu ngõ ra trungkhop chỉ bật lên mức 1 khi có lệnh yêu cầu so sánh cmd_sosanh = 1 VÀ mật khẩu đang nhập sau khi được xáo trộn (pass_16bit ^ mask) giống y hệt dữ liệu đã lưu trữ (stored_pass).

2.3. Module: fsm_controller.v

Giải đồ FSM:



- **Vai trò khối:** Là bộ não điều phối (Control Logic), quản lý dòng chảy hoạt động của toàn bộ hệ thống dựa trên mô hình Máy trạng thái hữu hạn (FSM).
- **Chức năng từng khối logic:** Đọc cờ ngõ vào (nút bấm, kết quả đối chiếu), quyết định trạng thái tiếp theo, xuất các lệnh (enable) điều khiển datapath, đếm ngược thời gian (timer) và đếm số lần sai mã.
- **Đoạn code phân tích (Chuyển đổi trạng thái cốt lõi):**

```

always @(posedge clk or negedge reset_n) begin
    if (!reset_n) current_state <= S_IDLE;
    else current_state <= next_state;
end

always @(*) begin
    next_state = current_state;
    case (current_state)
        S_IDLE: begin
            if (co_phim && (ma_phim <= 4'd9)) next_state = S_INPUT;
        end
        S_INPUT: begin
            if (co_phim) begin
                if (ma_phim == 4'hA) next_state = S_COMPARE;
                else if (ma_phim == 4'hC) next_state = S_IDLE;
            end
        end
        S_COMPARE: begin
            if (is_match) next_state = S_OPEN;
            else next_state = S_WRONG;
        end
        S_OPEN: begin
            if (timer >= 32'd250_000_000) next_state = S_IDLE;
            else if (co_phim && ma_phim == 4'hD) next_state = S_CHANGE_PASS;
        end
        S_WRONG: begin
            if (timer >= 32'd50_000_000) next_state = S_IDLE;
        end
        S_CHANGE_PASS: begin
            if (co_phim && ma_phim == 4'hA) next_state = S_IDLE;
        end
        default: next_state = S_IDLE;
    endcase
end
  
```

- **Giải thích chi tiết:**
 - Sử dụng thiết kế FSM chuẩn: Thanh ghi trạng thái cập nhật đồng bộ theo xung nhịp (`current_state <= next_state`) và mạch tổ hợp quyết định trạng

thái kế tiếp (always @*).

- Ví dụ luồng: Từ trạng thái S_IDLE (chờ), khi có phím bấm từ 0-9, chuyển sang S_INPUT. Trong lúc nhập, nếu bấm A (Enter), nhảy vào S_COMPARE (So sánh), từ đây phân nhánh nhờ cờ is_match để báo mở cửa (S_OPEN) hoặc báo sai (S_WRONG).

- **Đoạn code phân tích (Bộ định thời Timer & Đếm lỗi):**

```
S_WRONG: begin
    timer <= timer + 1;
    if (timer == 1) error_count <= error_count + 1; // Tăng đếm lỗi

    clear_buf <= 1; // Xóa bộ đệm ngay lập tức

end
```

Bộ Timer được tăng mỗi xung clock (50MHz). Ở trạng thái sai mã ('S_WRONG'), mạch cộng thêm 'error_count' và tự động kích hoạt 'clear_buf' để xóa toàn bộ số đang hiển thị trên màn hình. Mạch phát báo động khi 'assign alarm = (error_count >= 2'd3);' (nhập sai 3 lần).

3. Khối xử lý Output (Output & Display):

3.1. Module: led_7_doan.v (Giải mã hiển thị LED)

- **Vai trò khối:** Chuyển đổi mã logic nội bộ của hệ thống thành chuẩn tín hiệu điện để bật/tắt các đoạn sáng (segment) của LED vật lý trên board Altera DE2.
- **Chức năng từng khối logic:** Tách mã 16-bit thành 4 LED riêng biệt, giải mã mã Hex (BCD) sang tổ hợp sáng/tối của bóng LED.
- **Đoạn code phân tích:**

```
module led_7_doan (
    input wire [15:0] pass_16bit,
    output reg [6:0] HEX3,
    output reg [6:0] HEX2,
    output reg [6:0] HEX1,
    output reg [6:0] HEX0
);
    function [6:0] decode_7seg;
        input [3:0] bcd;
        begin
            case (bcd)
                4'h0: decode_7seg = 7'b1000000;
                4'h1: decode_7seg = 7'b11111001;
                4'h2: decode_7seg = 7'b0100100;
                4'h3: decode_7seg = 7'b0110000;
                4'h4: decode_7seg = 7'b0011001;
                4'h5: decode_7seg = 7'b0010010;
                4'h6: decode_7seg = 7'b0000010;
                4'h7: decode_7seg = 7'b11111000;
                4'h8: decode_7seg = 7'b0000000;
                4'h9: decode_7seg = 7'b0010000;
                4'hF: decode_7seg = 7'b1111111;
                default: decode_7seg = 7'b1111111;
            endcase
        end
    endfunction
```

Ánh xạ ra 4 LED:

```
always @(*) begin
    HEX3 = decode_7seg(pass_16bit[15:12]);
    HEX2 = decode_7seg(pass_16bit[11:8]);
    HEX1 = decode_7seg(pass_16bit[7:4]);
    HEX0 = decode_7seg(pass_16bit[3:0]);
end
endmodule
```

- **Giải thích chi tiết:** Module viết một hàm function giải mã chuẩn Anode Chung (Mức 0 là sáng). Điểm đáng chú ý trong code là việc bắt trường hợp 4'hF sẽ trả về 7'b1111111 (mức 1 tắt cả 7 đoạn), tạo cảm giác phím chưa được nhập thì đèn không sáng chữ số "0" dư thừa.

3.2. Module: warning_buzzer.v (Mạch cảnh báo ngoại vi)

- **Vai trò khối:** Điều khiển hệ thống phản hồi vật lý: Âm thanh (Còi/Buzzer) và Ánh sáng (LED báo hiệu).
- **Chức năng từng khối logic:** Bộ đếm chia tần số clock để tạo hiệu ứng nhấp nháy, kiểm tra còi (Mở cửa/Báo động) để xuất điện áp ra chân linh kiện.
- **Đoạn code phân tích:**

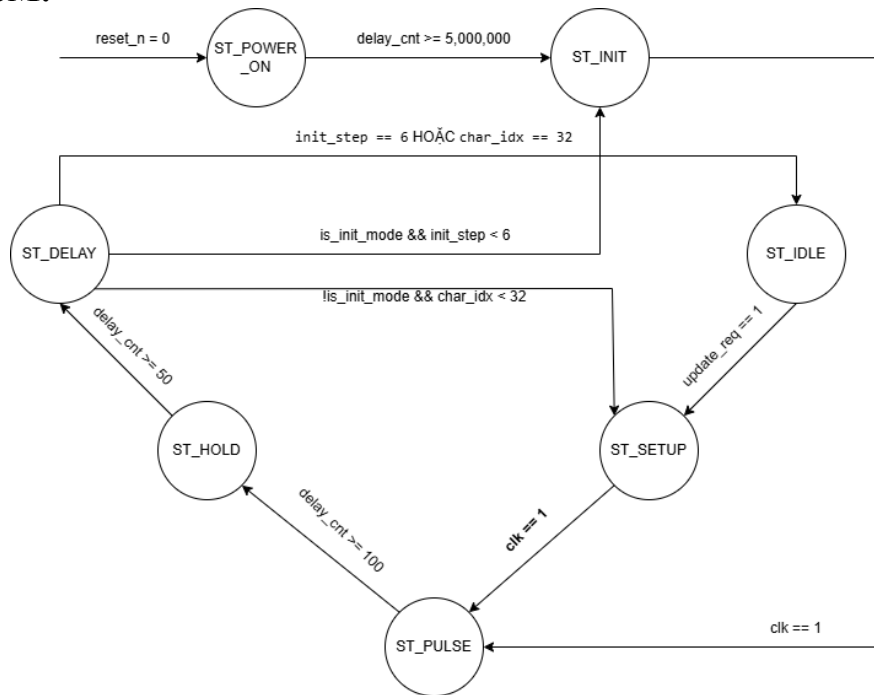
```
always @(posedge clk_50M or negedge reset_n) begin
    if (!reset_n) begin
        counter <= 24'd0;
        blink_sig <= 1'b0;
    end else if (counter >= 24'd12_499_999) begin // Đếm tới chu kỳ 0.25 giây
        counter <= 24'd0;
        blink_sig <= ~blink_sig; // Đảo trạng thái tạo xung chớp
    end else begin
        counter <= counter + 1'b1;
    end
end

always @(*) begin
    LEDG = 1'b0; // Mặc định tắt hết
    LEDR = 1'b0;
    BUZZER = 1'b0;
    if (mo_cua_ok) begin // Sáng tĩnh LED Xanh
        LEDG = 1'b1;
    end
    else if (sai_pass) begin
        LEDR = blink_sig; // LED Đỏ chớp tắt
        BUZZER = blink_sig; // Còi kêu theo nhịp
    end
end
endmodule
```

- **Giải thích chi tiết:** Tín hiệu blink_sig có chu kỳ xấp xỉ 2Hz (chớp 2 lần mỗi giây) dựa vào bộ đếm tràn mức 12.5 triệu xung của clock 50Mhz. Nếu sai quá số lần (sai_pass), LED Đỏ và Còi được gắn vào blink_sig để tạo hiệu ứng cảnh báo an ninh chớp tắt sinh động.

3.3. Module: lcd_controller.v (Trình điều khiển giao diện LCD 16x2)

Giải đồ FSM:



- **Vai trò khối:** Giao tiếp mạch số song song với chip HD44780 của màn hình LCD 16x2.
- **Chức năng từng khối logic:** Xây dựng thông điệp hiển thị (ASCII string) ứng với từng state_code từ FSM. Đồng thời chứa một FSM định thời siêu nhỏ bên trong để kích xung điều khiển.
- **Đoạn code phân tích (Tạo thông điệp):**

```

always @(*) begin
    case (state_code)
        S_IDLE: begin
            str_1 = "CHOOSE MODE: ";
            str_2 = "1:OPEN 0:CHANGE ";
        end
        S_INPUT: begin
            str_1 = "ENTER PASSWORD: ";
            str_2 = "***** ";
        end
        S_COMPARE: begin
            str_1 = "CHECKING... ";
            str_2 = "PLEASE WAIT ";
        end
        S_OPEN: begin
            str_1 = "DOOR OPENED ! ";
            str_2 = "ACCESS GRANTED ";
        end
        S_WRONG: begin
            str_1 = "WRONG PASS ! ";
            str_2 = "PRESS C TO RETRY";
        end
        S_CHANGE_PASS: begin
            str_1 = "ENTER NEW PASS: ";
            str_2 = "***** ";
        end
        default: begin
            str_1 = " ";
            str_2 = " ";
        end
    endcase
    // Cắt chuỗi lớn thành các ký tự 8-bit
    for (i = 0; i < 16; i = i + 1) begin
        lcd_line_1[i] = str_1[127 - (i * 8) -: 8];
        lcd_line_2[i] = str_2[127 - (i * 8) -: 8];
    end
end
  
```

- **Giải thích chi tiết:** Khi FSM tổng đổi trạng thái, khối lệnh tổ hợp này thay đổi chuỗi 32 byte dữ liệu thô (2 dòng x 16 ký tự). Vòng lặp for làm nhiệm vụ bóc tách từng ký tự 8-bit nhét vào mảng dữ liệu.
- **Đoạn code phân tích (Tạo xung điều khiển timing):**

```
PULSE_HIGH: begin
    LCD_EN <= 1'b1;
    if (delay_cnt >= 20'd50) begin
        delay_cnt <= 20'd0;
        lcd_state <= HOLD;
    end else begin
        delay_cnt <= delay_cnt + 1'b1;
    end
end
```

Giải thích chi tiết: Vì IC LCD HD44780 xử lý rất chậm so với FPGA 50MHz, module phải tự sinh ra các trạng thái SETUP (nạp dữ liệu), PULSE_HIGH (đưa chân LCD_EN lên mức 1 và trễ giữ mức này bằng delay_cnt để LCD kịp đọc), rồi qua HOLD, DELAY. Điều này đảm bảo LCD không hiển thị rác ký tự do việc "nhồi" dữ liệu quá nhanh

Chương VI. KIỂM TRA VÀ MÔ PHỎNG HỆ THỐNG

1. Mục tiêu kiểm tra

Sử dụng Testbench để xác minh tính đúng đắn của toàn bộ các chức năng logic trong hệ thống khóa số (Digital Lock), bao gồm:

- Khả năng nhận dạng phím từ ma trận bàn phím (Matrix Keypad).
- Logic so khớp mật khẩu và điều khiển cửa (Door Control).
- Chức năng đổi mật khẩu (Change Password).
- Các phím chức năng đặc biệt: Xóa lùi (Backspace) và Xóa hết (Clear).
- Logic bảo mật: Kích hoạt báo động khi sai mật khẩu nhiều lần.

2. Các công cụ hỗ trợ trong kịch bản (Helper Tasks)

Để quá trình mô phỏng diễn ra hiệu quả và nhanh chóng, kịch bản sử dụng hai kỹ thuật chính:

- Task press_key: Giả lập việc bấm phím vật lý. Task này tự động tính toán tọa độ Hàng/Cột (Row/Col) và duy trì tín hiệu đủ lâu (30ms) để vượt qua bộ lọc chống nhiễu (Debounce) của mạch.
- Task skip_timer (Kỹ thuật force/release): Vì trong thực tế, cửa mở 5 giây hay khóa 1 giây tương ứng với hàng triệu chu kỳ xung clock (rất nặng cho máy tính khi mô phỏng). Task này thực hiện "tua nhanh" thời gian bằng cách ép giá trị thanh ghi timer lên mức cao để chuyển trạng thái tức thì mà không cần đợi lâu.

3. Kịch bản các trường hợp kiểm thử (Test Scenarios)

Kịch bản 1: Kiểm tra mật khẩu mặc định (Default Password)

- Hành động: Nhập chuỗi số 0-0-0-0 và nhấn phím A (Enter).
- Mục tiêu: Xác minh hệ thống nhận diện đúng mật khẩu khởi tạo trong code (16'h0000).
- Kết quả mong đợi: Tín hiệu LEDG (Đèn xanh) bật sáng, hệ thống chuyển sang trạng thái S_OPEN.

Kịch bản 2: Quy trình đổi mật khẩu mới (Change Password)

- Hành động: 1. Nhập mật khẩu cũ để mở cửa. 2. Khi cửa đang mở, nhấn phím D để vào chế độ đổi mật khẩu. 3. Nhập mật khẩu mới là 1-2-3-4 và nhấn A để lưu.
- Mục tiêu: Kiểm tra khả năng cập nhật vùng nhớ mật khẩu trong khối passdatapath.
- Kết quả mong đợi: Hệ thống lưu mật khẩu mới thành công và quay về trạng thái khóa.

Kịch bản 3: Xác minh mật khẩu sau khi đổi (Verification)

- Hành động: Nhập lại mật khẩu cũ (0-0-0-0).
- Mục tiêu: Đảm bảo mật khẩu cũ không còn tác dụng sau khi đã đổi.

- Kết quả mong đợi: Hệ thống báo lỗi (LEDR nháy), trạng thái chuyển sang S_WRONG.

Kịch bản 4: Kiểm tra phím chức năng Backspace (Xóa lùi)

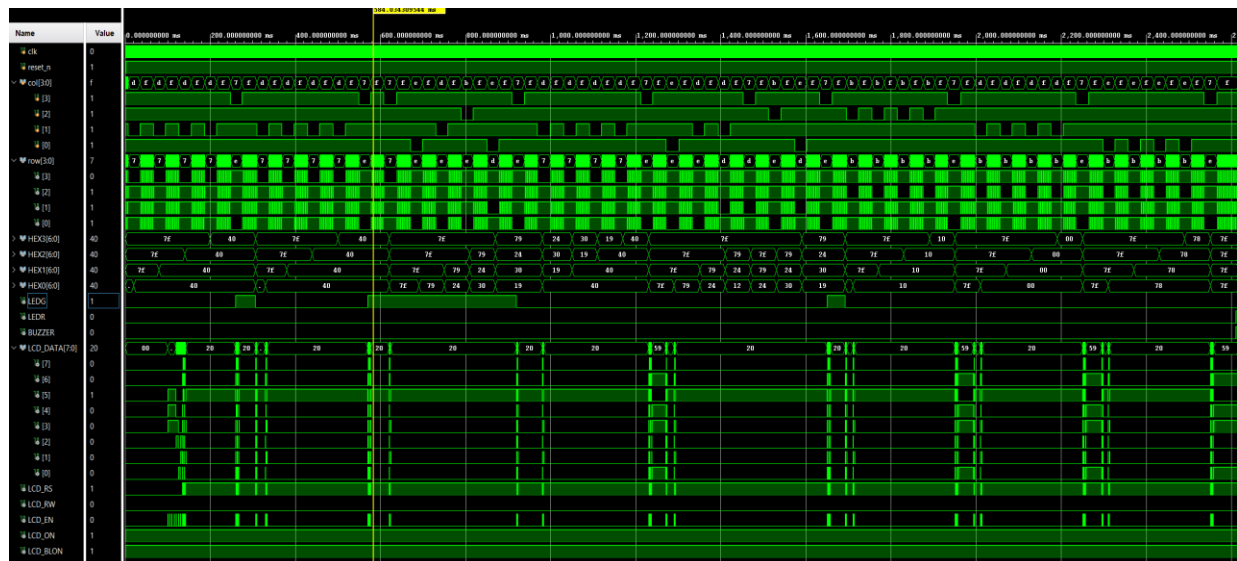
- Hành động: Nhập chuỗi 1-2-5, sau đó nhấn phím B (Backspace) để xóa số 5, rồi nhập tiếp 3-4 và nhấn A.
- Mục tiêu: Kiểm tra logic dịch chuyển phải (Shift Right) của Buffer khi người dùng nhập sai một ký tự.
- Kết quả mong đợi: Hệ thống nhận diện đúng mật khẩu cuối cùng là 1-2-3-4 và mở cửa thành công.

Kịch bản 5: Kiểm tra tính năng bảo mật (Alarm System)

- Hành động: Cố tình nhập sai mật khẩu liên tiếp 3 lần (ví dụ: 9999, 8888, 7777).
- Mục tiêu: Xác minh bộ đếm lỗi error_count và tín hiệu báo động.
- Kết quả mong đợi: Sau lần sai thứ 3, tín hiệu BUZZER (Còi) kích hoạt lên mức cao, hệ thống báo động.

4. Mô phỏng:

4.1. Waveform và console:



```

Tcl Console
run all

--- TEST 1: NHAP DUNG MAT KHAU MAC DINH ---
[5001000000] Nhan Phim: 0
[65001000000] Nhan Phim: 0
[125001000000] Nhan Phim: 0
[185001000000] Nhan Phim: 0
[245001000000] Nhan Phim: a
-> PASS! Cua da mo (LEDG = 1)
[306001000000] ---> TUA NHANH THOI GIAN DONG CUA (5 giay) <---

--- TEST 2: DOI MAT KHAU (Moi: 1234) ---
[307051000000] Nhan Phim: 0
[367051000000] Nhan Phim: 0
[427051000000] Nhan Phim: 0
[487051000000] Nhan Phim: 0
[547051000000] Nhan Phim: a
[608051000000] Nhan Phim: d
Dang nhap pass moi...
[668051000000] Nhan Phim: 1
[728051000000] Nhan Phim: 2
[788051000000] Nhan Phim: 3
[848051000000] Nhan Phim: 4
[908051000000] Nhan Phim: a
[968051000000] ---> TUA NHANH THOI GIAN DONG CUA (5 giay) <---

--- TEST 3: THU MO CUA BANG PASS CU (0000) ---
[969101000000] Nhan Phim: 0
[1029101000000] Nhan Phim: 0
[1089101000000] Nhan Phim: 0
[1149101000000] Nhan Phim: 0
[1209101000000] Nhan Phim: a
-> PASS! He thong bao sai pass (Dung logic)
[1270101000000] ---> TUA NHANH THOI GIAN KHOA SAI PASS (1 giay) <---

--- TEST 4: TEST BACKSPACE (B) VA PASS MOI (1234) ---
[1271151000000] Nhan Phim: 1
[1331151000000] Nhan Phim: 2
[1391151000000] Nhan Phim: 5
[1451151000000] Nhan Phim: b
[1511151000000] Nhan Phim: 3
[1571151000000] Nhan Phim: 4
[1631151000000] Nhan Phim: a
-> PASS! Mo cua thanh cong bang pass moi 1234, Backspace hoat dong tot!
[1692151000000] ---> TUA NHANH THOI GIAN DONG CUA (5 giay) <---

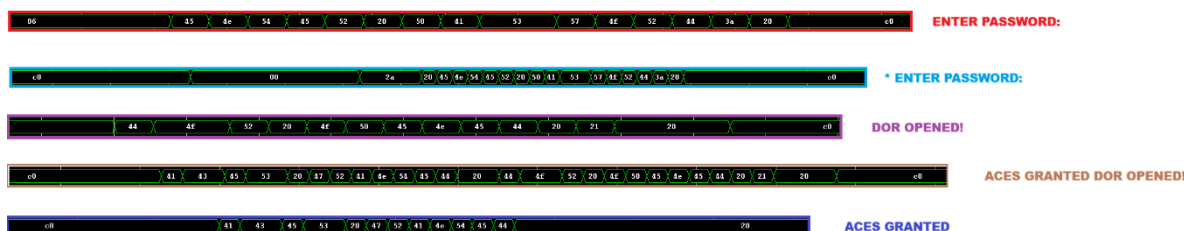
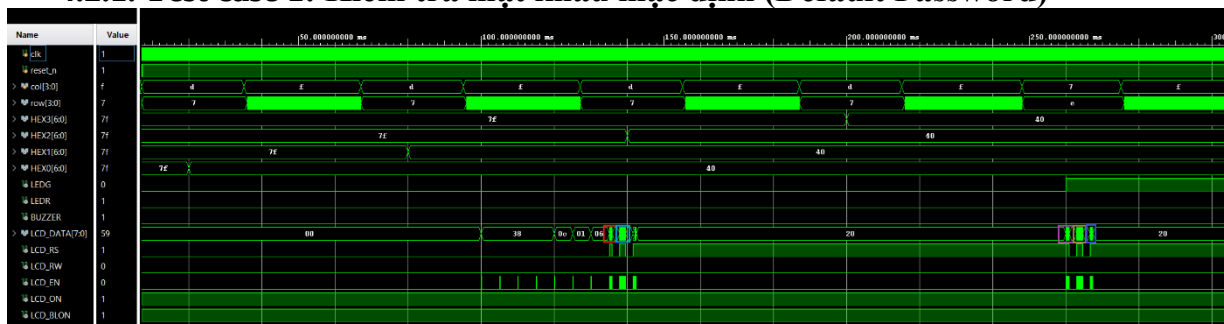
--- TEST 5: TEST HE THONG CANH BAO BUZZER (Sai 3 lan) ---
[1693201000000] Nhan Phim: 9
[1753201000000] Nhan Phim: 9
[1813201000000] Nhan Phim: 9
[1873201000000] Nhan Phim: 9
[1933201000000] Nhan Phim: a
[1993201000000] ---> TUA NHANH THOI GIAN KHOA SAI PASS (1 giay) <---
[1994251000000] Nhan Phim: 8
[2054251000000] Nhan Phim: 8
[2114251000000] Nhan Phim: 8
[2174251000000] Nhan Phim: 8
[2234251000000] Nhan Phim: a
[2294251000000] ---> TUA NHANH THOI GIAN KHOA SAI PASS (1 giay) <---
[2295301000000] Nhan Phim: 7
[2355301000000] Nhan Phim: 7
[2415301000000] Nhan Phim: 7
[2475301000000] Nhan Phim: 7
[2535301000000] Nhan Phim: a
-> FAIL! Coi khong keu!

==== HOAN THANH TESTBENCH ====
$finish called at time : 2616301 us : File "E:/UIT_Document/HDL_Verilog/Do
run: Time (s): cpu = 00:05:15 ; elapsed = 00:20:56 . Memory (MB): peak = 3
Type a Tcl command here

```

4.2. Phân tích mô phỏng:

4.2.1. Test case 1: Kiểm tra mật khẩu mặc định (Default Password)



```
--- TEST 1: NHAP DUNG MAT KHAU MAC DINH ---  
[5001000000] Nhan Phim: 0  
[65001000000] Nhan Phim: 0  
[125001000000] Nhan Phim: 0  
[185001000000] Nhan Phim: 0  
[245001000000] Nhan Phim: a  
-> PASS! Cua da mo (LEDG = 1)
```

Giai đoạn khởi tạo (0ms - 50ms)

- Reset hệ thống: Ngay khi bắt đầu, tín hiệu reset_n xuống mức 0 để đưa các thanh ghi về trạng thái mặc định. Sau đó reset_n lên 1 để mạch bắt đầu chạy.
- Khởi tạo LCD: Mạch chuyển qua các trạng thái ST_POWER_ON và ST_INIT. Các mã lệnh Hex như 38 (cấu hình 8-bit), 0C (bật hiển thị), 01 (xóa màn hình) được gửi qua chân LCD_DATA.
- Thông báo ban đầu: Màn hình LCD hiển thị dòng chữ "ENTER PASSWORD:". Trong hình cuối, có thể thấy chuỗi Hex tương ứng: 45 (E), 4E (N), 54 (T), 45 (E), 52 (R)... được gửi đi.

Quá trình nhập phím (Key Input)

- 500ms: Bấm phím '0' lần 1. (task sẽ tra bảng để biết số 0 nằm ở target_row = 3 và target_col = 1.)
- 650ms: Bấm phím '0' lần 2.
- 1250ms: Bấm phím '0' lần 3.
- 1850ms: Bấm phím '0' lần 4.
- 2450ms: Bấm phím 'A' (tương đương nút Enter) để xác nhận.

Kiểm tra mật khẩu và Kết quả

- So khớp logic: Khi phím 'A' được nhấn, máy trạng thái chuyển sang S_COMPARE. Logic bên trong so sánh chuỗi vừa nhập (0000) với mật khẩu mặc định được lưu trong bộ nhớ.
- Mở cửa (Access Granted): Do mật khẩu trùng khớp, tín hiệu LEDG chuyển từ 0

- lên 1 (Đèn xanh sáng), báo hiệu cửa đã mở.
- Tín hiệu BUZZER vẫn ở mức 0 (không báo động).
- Cập nhật LCD: Màn hình LCD thay đổi nội dung để thông báo trạng thái mới. Dựa trên phân tích Hex trong hình 5:
- Dòng 1: "ACCESS GRANTED" (Mã Hex: 41, 43, 45, 53...).
- Dòng 2: "DOOR OPENED!" (Mã Hex: 44, 4F, 52...).

4.2.2. Test case 2: Quy trình đổi mật khẩu mới (Change Password)



```

--- TEST 2: DOI MAT KHAU (Moi: 1234) ---
[307051000000] Nhan Phim: 0
[367051000000] Nhan Phim: 0
[427051000000] Nhan Phim: 0
[487051000000] Nhan Phim: 0
[547051000000] Nhan Phim: a
[608051000000] Nhan Phim: d
Dang nhap pass moi...
[668051000000] Nhan Phim: 1
[728051000000] Nhan Phim: 2
[788051000000] Nhan Phim: 3
[848051000000] Nhan Phim: 4
[908051000000] Nhan Phim: a
[968051000000] ---> TUA NHANH THOI GIAN DONG CUA (5 giay) <---
```

Giai đoạn 1: Đăng nhập bằng Pass cũ để mở cửa

- Để bảo mật, hệ thống chỉ cho phép đổi mật khẩu khi cửa đang ở trạng thái mở (S_OPEN).
- Hành động: Từ mốc thời gian 307ms đến 547ms, người dùng nhập mật khẩu mặc định là 0000 và nhấn phím A (Enter).
- Kết quả: Hệ thống xác nhận đúng mật khẩu. Tín hiệu LEDG (Đèn xanh) bật sáng.
- LCD phản hồi: Màn hình hiển thị dòng chữ "ACCESS GRANTED DOOR OPENED!" .

Giai đoạn 2: Kích hoạt chế độ đổi mật khẩu

- Hành động: Tại mốc 608ms, khi cửa vẫn đang mở, người dùng nhấn phím D (Mã chức năng đổi mật khẩu).
- Chuyển trạng thái: FSM chuyển từ S_OPEN sang trạng thái S_CHANGE_PASS.
- LCD phản hồi: Màn hình cập nhật dòng thông báo "ENTER NEW PASS:" để hướng dẫn người dùng.

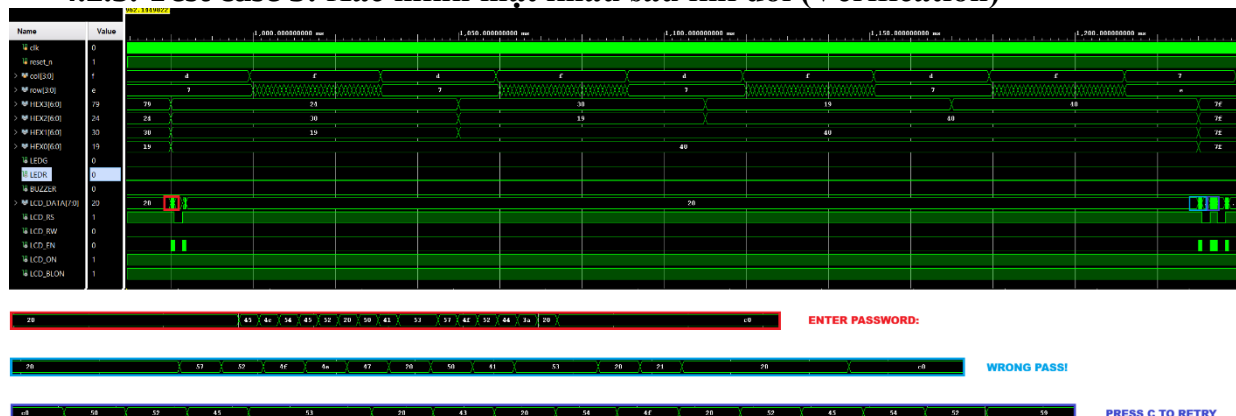
Giai đoạn 3: Nhập mật khẩu mới (1234)

- Hành động: Người dùng lần lượt nhập các phím 1, 2, 3, 4 (từ mốc 668ms đến 848ms).
- Xác nhận: Nhấn phím A (Enter) tại mốc 908ms để lưu giá trị này vào thanh ghi mật khẩu chính của hệ thống.
- Xử lý dữ liệu: Lúc này, khối passdatapath sẽ cập nhật giá trị mật khẩu từ 16'h0000 thành 16'h1234.

Giai đoạn 4: Kết thúc và Khóa hệ thống

- Tua nhanh thời gian: Tại mốc 968ms, Testbench gọi Task skip_door_timer để bỏ qua 5 giây chờ đóng cửa thực tế (nhằm tiết kiệm thời gian mô phỏng).
- Trạng thái cuối: Hệ thống tự động quay về trạng thái nghỉ (S_IDLE).
- LCD phản hồi: Màn hình quay lại trạng thái chờ ban đầu là "SYSTEM LOCKED" và "INPUT PASSWORD".

4.2.3. Test case 3: Xác minh mật khẩu sau khi đổi (Verification)



```

--- TEST 3: THU MO CUA BANG PASS CU (0000) ---
[969101000000] Nhan Phim: 0
[1029101000000] Nhan Phim: 0
[1089101000000] Nhan Phim: 0
[1149101000000] Nhan Phim: 0
[1209101000000] Nhan Phim: a
-> PASS! He thong bao sai pass (Dung logic)

```

Diễn biến thời gian và hành động

- LCD hiển thị "ENTER PASSWORD:" - Hệ thống sẵn sàng nhận mã.
- Mốc 969ms - 1149ms: Người dùng nhập liên tiếp 4 phím '0'.
- Mốc 1209ms: Người dùng nhấn phím 'a' (Enter) để xác nhận.
- LCD hiển thị "WRONG PASS!" - Thông báo mật khẩu không chính xác ngay

- Kết luận của Console: -> PASS! Hệ thống bao sai pass (Dùng logic). Điều này có nghĩa là hệ thống đã nhận diện chính xác đây là một nỗ lực truy cập trái phép bằng mật khẩu đã hết hạn.

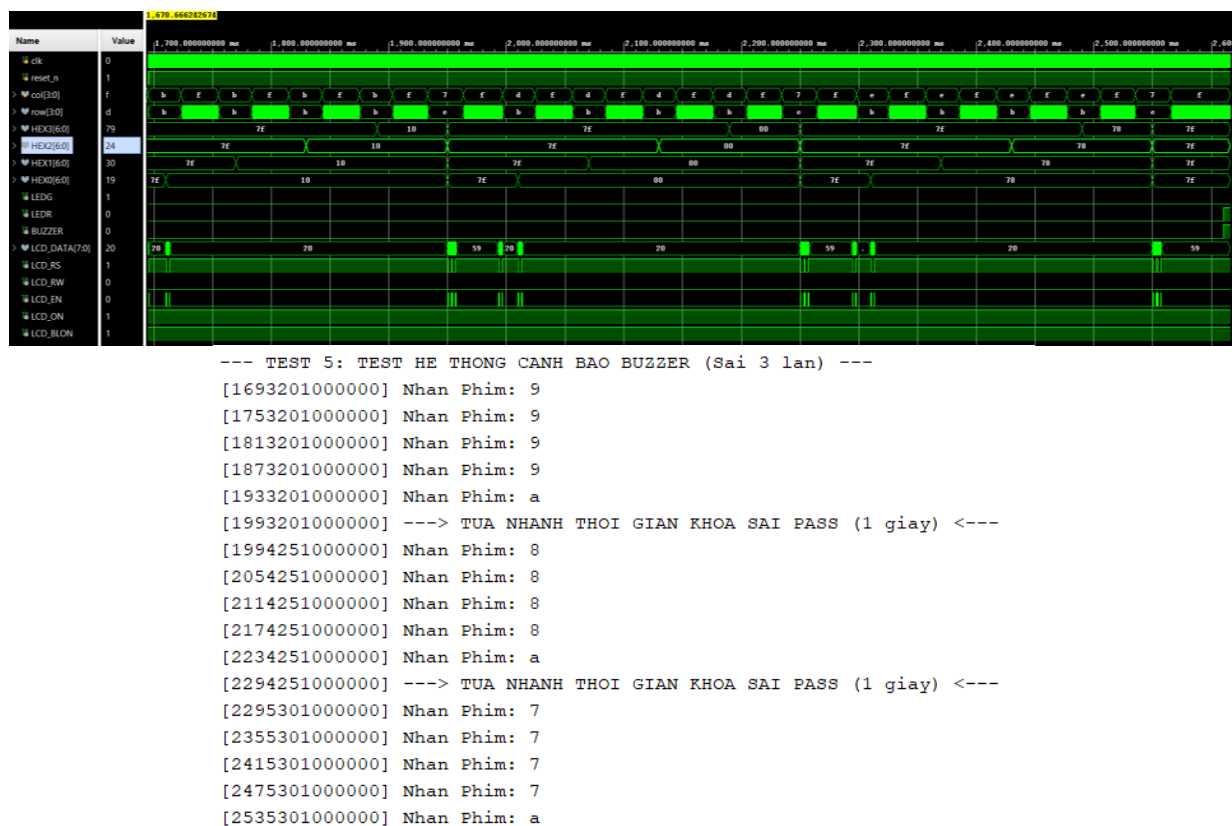
- Khi phím 'a' được nhấn, chúng ta thấy sự thay đổi rõ rệt trên biểu đồ sóng (Waveform):
- Tín hiệu LEDR (Red LED): Chuyển từ 0 lên 1. Đèn đỏ sáng lên báo hiệu nhập sai.
- Tín hiệu LEDG (Green LED): Giữ nguyên ở mức 0. Cửa vẫn khóa chặt.
- Tín hiệu BUZZER: chưa được kéo lên 1 vì theo logic bấm sai pass 3 lần mới kích hoạt báo động.

The timing diagram illustrates the operation of the LCD module. The signals are as follows:

- CLK**: Clock signal, shown as a series of pulses.
- reset_n**: Active-low reset signal, shown as a single pulse.
- col[20]**: Column address bus, showing values 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20.
- row[30]**: Row address bus, showing values 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30.
- HDX[6:0]**: Horizontal sync signal, shown as a series of pulses.
- LEDG**: Green LED enable signal, shown as a series of pulses.
- LEDR**: Red LED enable signal, shown as a series of pulses.
- BUZZER**: Buzzer enable signal, shown as a series of pulses.
- LCD_DATA[7:0]**: 8-bit data bus, showing the sequence of data being sent to the LCD. The data includes the command 0x20 (clear display) and the character 'a' (0x61).
- LCD_RS**: Register select signal, shown as a series of pulses.
- LCD_RW**: Read/Write signal, shown as a series of pulses.
- LCD_EN**: Enable signal, shown as a series of pulses.
- LCD_ON**: On signal, shown as a series of pulses.
- LCD_BLON**: Backlight on signal, shown as a series of pulses.

- Quét phím: Tín hiệu row và col thay đổi liên tục tương ứng với mỗi lần nhấn phím trong log (đầu tiên là 1 2 5 sau đó nhấn phím B (xóa) và nhấn tiếp 3 4 rồi nhấn A, chứng tỏ khối quét ma trận phím nhận diện đúng các phím chức năng ('b' và 'a').
- Kết quả xác thực: Ngay sau khi nhấn phím 'a' tại mốc 1631ms, hệ thống chuyển trạng thái:
- LEDG (Green LED): Bật lên mức 1, báo hiệu cửa mở thành công.
- LEDR (Red LED): Giữ ở mức 0.
- LCD DATA: Thực hiện gửi chuỗi dữ liệu mới để thông báo mở cửa.

4.2.5. Test case 5: Kiểm tra tính năng bảo mật (Alarm System)



Kịch bản nhập liệu (Console Log)

Hành động được thực hiện liên tục qua 3 lần thử sai:

- Lần 1 (1693ms - 1933ms): Nhập mật khẩu 9999, sau đó nhấn phím 'a' (Enter).
- Lần 2 (1994ms - 2234ms): Nhập mật khẩu 8888, sau đó nhấn phím 'a' (Enter).
- Lần 3 (2295ms - 2535ms): Nhập mật khẩu 7777, sau đó nhấn phím 'a' (Enter).
- Sau mỗi lần nhấn Enter sai, hệ thống thực hiện TUA NHANH THOI GIAN KHOA SAI PASS (1 giây) để chuyển sang lần nhập tiếp theo.

Phản hồi của Máy trạng thái (FSM)

- Mỗi lần nhấn 'a' với mật khẩu sai, điều kiện else tại S_COMPARE được kích hoạt, đưa hệ thống vào trạng thái S_WRONG.
- Tại S_WRONG, hệ thống kích hoạt bộ đếm thời gian. Sau khi timer $\geq 50.000.000$ (1 giây), FSM tự động quay về S_IDLE để người dùng nhập lại.
- Cơ chế đếm lỗi: Mặc dù không hiển thị trực tiếp trong đoạn mã FSM, nhưng một bộ đếm số lần sai (wrong_count) sẽ tăng dần sau mỗi lần trạng thái S_WRONG được kích hoạt.

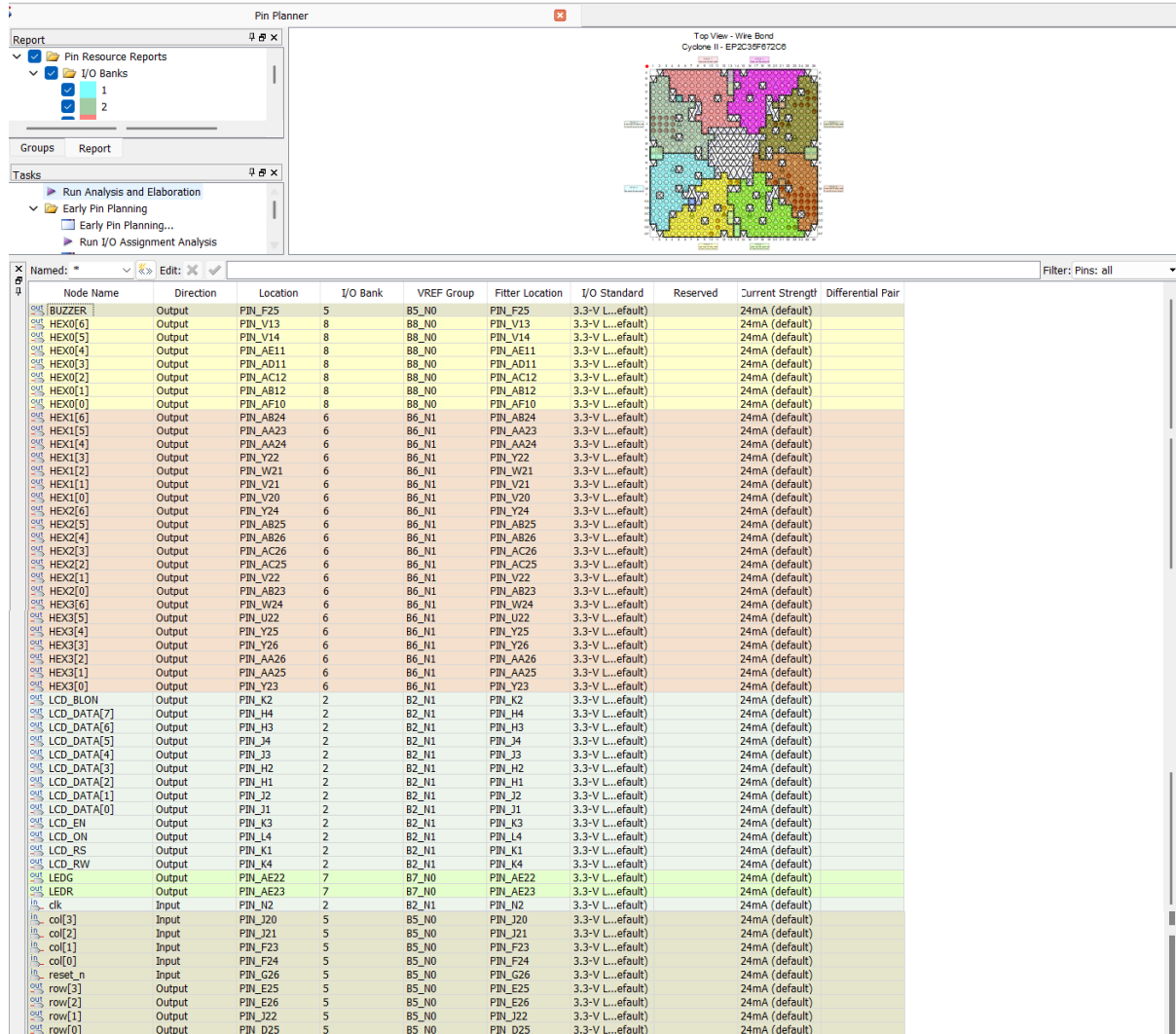
Phân tích tín hiệu Waveform

- Tín hiệu BUZZER: Chuyển từ mức 0 lên mức 1 (High). Đây là thời điểm hệ thống chính thức phát báo động vì đã vượt quá giới hạn cho phép.
- Tín hiệu LEDR (Red LED): Giữ ở mức cao hoặc nhấp nháy (tùy thiết kế) để cảnh báo trạng thái nguy hiểm.
- Tín hiệu LEDG (Green LED): Giữ nguyên ở mức 0, đảm bảo cửa không mở dù

người dùng cố gắng nhập nhiều lần.

- LCD_DATA: Gửi chuỗi ký tự thông báo hệ thống bị khóa ("SYSTEM LOCKED").

5. Gắn Pin:



The screenshot displays the Pin Planner interface for a Cyclone II EP2C35F10C08 device. The top view shows the chip layout with pins numbered 1 through 100. The main table lists the pin assignments for various components.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Differential Pair
[BUZZER]	Output	PIN_F25	5	B5_N0	PIN_F25	3.3-V L...efault		24mA (default)	
HEX0[6]	Output	PIN_V13	8	B8_N0	PIN_V13	3.3-V L...efault		24mA (default)	
HEX0[5]	Output	PIN_V14	8	B8_N0	PIN_V14	3.3-V L...efault		24mA (default)	
HEX0[4]	Output	PIN_AE11	8	B8_N0	PIN_AE11	3.3-V L...efault		24mA (default)	
HEX0[3]	Output	PIN_AD11	8	B8_N0	PIN_AD11	3.3-V L...efault		24mA (default)	
HEX0[2]	Output	PIN_AC12	8	B8_N0	PIN_AC12	3.3-V L...efault		24mA (default)	
HEX0[1]	Output	PIN_AB12	8	B8_N0	PIN_AB12	3.3-V L...efault		24mA (default)	
HEX0[0]	Output	PIN_AF10	8	B8_N0	PIN_AF10	3.3-V L...efault		24mA (default)	
HEX1[6]	Output	PIN_AB24	6	B6_N1	PIN_AB24	3.3-V L...efault		24mA (default)	
HEX1[5]	Output	PIN_AA23	6	B6_N1	PIN_AA23	3.3-V L...efault		24mA (default)	
HEX1[4]	Output	PIN_AA24	6	B6_N1	PIN_AA24	3.3-V L...efault		24mA (default)	
HEX1[3]	Output	PIN_Y22	6	B6_N1	PIN_Y22	3.3-V L...efault		24mA (default)	
HEX1[2]	Output	PIN_W21	6	B6_N1	PIN_W21	3.3-V L...efault		24mA (default)	
HEX1[1]	Output	PIN_V21	6	B6_N1	PIN_V21	3.3-V L...efault		24mA (default)	
HEX1[0]	Output	PIN_V20	6	B6_N1	PIN_V20	3.3-V L...efault		24mA (default)	
HEX2[6]	Output	PIN_Y24	6	B6_N1	PIN_Y24	3.3-V L...efault		24mA (default)	
HEX2[5]	Output	PIN_AB25	6	B6_N1	PIN_AB25	3.3-V L...efault		24mA (default)	
HEX2[4]	Output	PIN_AB26	6	B6_N1	PIN_AB26	3.3-V L...efault		24mA (default)	
HEX2[3]	Output	PIN_AC26	6	B6_N1	PIN_AC26	3.3-V L...efault		24mA (default)	
HEX2[2]	Output	PIN_AC25	6	B6_N1	PIN_AC25	3.3-V L...efault		24mA (default)	
HEX2[1]	Output	PIN_V22	6	B6_N1	PIN_V22	3.3-V L...efault		24mA (default)	
HEX2[0]	Output	PIN_AB23	6	B6_N1	PIN_AB23	3.3-V L...efault		24mA (default)	
HEX3[6]	Output	PIN_W24	6	B6_N1	PIN_W24	3.3-V L...efault		24mA (default)	
HEX3[5]	Output	PIN_U22	6	B6_N1	PIN_U22	3.3-V L...efault		24mA (default)	
HEX3[4]	Output	PIN_Y25	6	B6_N1	PIN_Y25	3.3-V L...efault		24mA (default)	
HEX3[3]	Output	PIN_Y26	6	B6_N1	PIN_Y26	3.3-V L...efault		24mA (default)	
HEX3[2]	Output	PIN_AA26	6	B6_N1	PIN_AA26	3.3-V L...efault		24mA (default)	
HEX3[1]	Output	PIN_AA25	6	B6_N1	PIN_AA25	3.3-V L...efault		24mA (default)	
HEX3[0]	Output	PIN_Y23	6	B6_N1	PIN_Y23	3.3-V L...efault		24mA (default)	
LCD_BLON	Output	PIN_K2	2	B2_N1	PIN_K2	3.3-V L...efault		24mA (default)	
LCD_DATA[7]	Output	PIN_H4	2	B2_N1	PIN_H4	3.3-V L...efault		24mA (default)	
LCD_DATA[6]	Output	PIN_H3	2	B2_N1	PIN_H3	3.3-V L...efault		24mA (default)	
LCD_DATA[5]	Output	PIN_J4	2	B2_N1	PIN_J4	3.3-V L...efault		24mA (default)	
LCD_DATA[4]	Output	PIN_J3	2	B2_N1	PIN_J3	3.3-V L...efault		24mA (default)	
LCD_DATA[3]	Output	PIN_H2	2	B2_N1	PIN_H2	3.3-V L...efault		24mA (default)	
LCD_DATA[2]	Output	PIN_H1	2	B2_N1	PIN_H1	3.3-V L...efault		24mA (default)	
LCD_DATA[1]	Output	PIN_J2	2	B2_N1	PIN_J2	3.3-V L...efault		24mA (default)	
LCD_DATA[0]	Output	PIN_J1	2	B2_N1	PIN_J1	3.3-V L...efault		24mA (default)	
LCD_EN	Output	PIN_K3	2	B2_N1	PIN_K3	3.3-V L...efault		24mA (default)	
LCD_ON	Output	PIN_L4	2	B2_N1	PIN_L4	3.3-V L...efault		24mA (default)	
LCD_RS	Output	PIN_K1	2	B2_N1	PIN_K1	3.3-V L...efault		24mA (default)	
LCD_RW	Output	PIN_K4	2	B2_N1	PIN_K4	3.3-V L...efault		24mA (default)	
LEDg	Output	PIN_AE22	7	B7_N0	PIN_AE22	3.3-V L...efault		24mA (default)	
LEDR	Output	PIN_AE23	7	B7_N0	PIN_AE23	3.3-V L...efault		24mA (default)	
clk	Input	PIN_H2	2	B2_N1	PIN_H2	3.3-V L...efault		24mA (default)	
col[3]	Input	PIN_J20	5	B5_N0	PIN_J20	3.3-V L...efault		24mA (default)	
col[2]	Input	PIN_J21	5	B5_N0	PIN_J21	3.3-V L...efault		24mA (default)	
col[1]	Input	PIN_F23	5	B5_N0	PIN_F23	3.3-V L...efault		24mA (default)	
col[0]	Input	PIN_F24	5	B5_N0	PIN_F24	3.3-V L...efault		24mA (default)	
reset_n	Input	PIN_G26	5	B5_N0	PIN_G26	3.3-V L...efault		24mA (default)	
row[3]	Output	PIN_E25	5	B5_N0	PIN_E25	3.3-V L...efault		24mA (default)	
row[2]	Output	PIN_E26	5	B5_N0	PIN_E26	3.3-V L...efault		24mA (default)	
row[1]	Output	PIN_J22	5	B5_N0	PIN_J22	3.3-V L...efault		24mA (default)	
row[0]	Output	PIN_D25	5	B5_N0	PIN_D25	3.3-V L...efault		24mA (default)	

6. Test trực tiếp trên Kit DE2:

Video: <https://drive.google.com/file/d/1TUdA8HildnyS-GQhTlJvYWO3OpwLcJ-s/view?usp=drive>

CHƯƠNG VII: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và thực thi hệ thống trên kit Altera FPGA DE2, nhóm đã hoàn thành các mục tiêu đề ra ban đầu của đề án. Cụ thể:

- **Về phần cứng:** Đã kết nối thành công và vận hành ổn định bàn phím ma trận 4x4 cùng màn hình LCD 16x2 thông qua các chân GPIO trên FPGA.
- **Về lập trình (HDL):** Xây dựng thành công kiến trúc phân tầng bao gồm các khối:
 - **Khối Input:** Xử lý chống nhiễu phím (debouncing) và quét ma trận chính xác.
 - **Khối Logic:** Thiết kế máy trạng thái hữu hạn (FSM) điều khiển toàn bộ luồng hoạt động của khóa. Thực hiện thành công thuật toán mã hóa mật khẩu bằng kỹ thuật XOR Masking ngay tại mức phần cứng.
 - **Khối Output:** Hiển thị trạng thái trực quan trên LCD, LED 7 đoạn và cảnh báo bằng âm thanh qua Buzzer.
- **Về tính năng:** Hệ thống thực hiện đầy đủ các chức năng: Kiểm tra mật khẩu, mở cửa tự động đóng sau 5 giây, đổi mật khẩu linh hoạt và báo động khi nhập sai quá 3 lần.

2. Đánh giá ưu điểm và hạn chế

2.1. Ưu điểm

- **Tính bảo mật cao:** Nhờ việc xử lý song song của FPGA, các thao tác kiểm tra mật khẩu diễn ra tức thời. Việc sử dụng kỹ thuật XOR Masking giúp bảo vệ mật khẩu tránh bị truy xuất trực tiếp từ các thanh ghi bộ nhớ.
- **Giao diện thân thiện:** Sự kết hợp giữa LCD (hướng dẫn bằng chữ) và LED 7 đoạn (hiển thị số đang nhập) giúp người dùng dễ dàng thao tác mà không cần hướng dẫn phức tạp.
- **Độ ổn định:** Hệ thống hoạt động tin cậy, không xảy ra hiện tượng treo hoặc treo máy nhờ kiến trúc xử lý luồng (pipeline) và quản lý xung nhịp hệ thống chặt chẽ.

2.2. Hạn chế

- **Tính lưu trữ:** Hiện tại mật khẩu chỉ được lưu trữ trên các thanh ghi (registers) của FPGA, dẫn đến việc mật khẩu sẽ quay về mặc định (0000) nếu hệ thống bị mất nguồn điện.
- **Tính cơ động:** Hệ thống hiện đang chạy trên bộ Kit phát triển có kích thước lớn, chưa được tối ưu hóa về mặt diện tích để lắp đặt vào các cánh cửa thực tế.

3. Hướng phát triển đề tài

Trong tương lai, để hệ thống hoàn thiện và mang tính ứng dụng thực tiễn cao hơn, nhóm đề xuất một số hướng cải tiến như sau:

- **Lưu trữ không mất dữ liệu:** Tích hợp giao tiếp với bộ nhớ EEPROM hoặc Flash trên Board để lưu mật khẩu vĩnh viễn ngay cả khi ngắt nguồn.
- **Mở rộng phương thức xác thực:** Kết hợp thêm các module nhận diện vân tay (Fingerprint) hoặc thẻ từ (RFID) để tạo thành hệ thống bảo mật đa lớp.
- **Kết nối thông minh (IoT):** Tích hợp module Wi-Fi (như ESP8266/ESP32) để gửi thông báo về điện thoại khi có người nhập sai mật khẩu quá nhiều lần hoặc cho phép mở khóa từ xa qua ứng dụng.
- **Tối ưu hóa năng lượng:** Thiết kế chế độ "Sleep Mode" cho FPGA khi không có thao tác phím để tiết kiệm năng lượng cho các hệ thống dùng pin.

NHẬN XÉT VÀ ĐÁNH GIÁ

Đồ án "Khóa số sử dụng FPGA" đã giúp nhóm củng cố kiến thức về ngôn ngữ lập trình phân cứng Verilog, hiểu rõ hơn về quy trình thiết kế hệ thống số và cách thức giao tiếp với các thiết bị ngoại vi. Dù còn một số hạn chế về mặt lưu trữ năng lượng và kích thước, nhưng đây là nền tảng quan trọng để nhóm tiếp tục phát triển các ứng dụng bảo mật phức tạp hơn trong tương lai.

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và tên - MSSV	Công việc	Phần trăm đóng góp
Võ Gia Huy - 23520654	Thiết kế toàn bộ khối xử lý output, lên kịch bản thiết kế kiểm tra mô phỏng trên ModelSim và Kit DE2, viết 40% báo cáo đồ án và 20% slide báo cáo (có thuyết trình báo cáo đồ án).	40%
Nguyễn Đức Khang - 23520695	Thiết kế toàn bộ khối xử lý input, khối xử lý logic XOR Masking, viết 40% báo cáo đồ án và 80% slide báo cáo (có thuyết trình báo cáo đồ án).	40%
Nguyễn Vương Hiếu - 23520488	Debug cho khối xử lý logic, viết 20% báo cáo đồ án.	20%